

LLMorpheus: Mutation Testing Using Large Language Models

Frank Tip , Senior Member, IEEE, Jonathan Bell , and Max Schäfer 

Abstract—In mutation testing, the quality of a test suite is evaluated by introducing faults into a program and determining whether the program’s tests detect them. Most existing approaches for mutation testing involve the application of a fixed set of mutation operators, e.g., replacing a “+” with a “-”, or removing a function’s body. However, certain types of real-world bugs cannot easily be simulated by such approaches, limiting their effectiveness. This paper presents a technique for mutation testing where placeholders are introduced at designated locations in a program’s source code and where a Large Language Model (LLM) is prompted to ask what they could be replaced with. The technique is implemented in *LLMorpheus*, a mutation testing tool for JavaScript, and evaluated on 13 subject packages, considering several variations on the prompting strategy, and using several LLMs. We find *LLMorpheus* to be capable of producing mutants that resemble existing bugs that cannot be produced by *StrykerJS*, a state-of-the-art mutation testing tool. Moreover, we report on the running time, cost, and number of mutants produced by *LLMorpheus*, demonstrating its practicality.

Index Terms—Mutation testing, large language models.

I. INTRODUCTION

MUTATION testing is an approach for evaluating the adequacy of a test suite and is increasingly adopted in industrial settings [1], [2], [3]. With mutation testing, an automated tool repeatedly injects a small modification to the system under test and executes the test suite on this mutated code. Mutation testing is premised on the *competent programmer hypothesis*, which posits that most buggy programs are quite close to being correct and that complex faults are *coupled* with simpler faults [4], i.e., a test that is strong enough to detect a simple fault should also be able to detect a more complex one. Hence, mutation analysis tools typically apply a relatively small set of mutation operators: replacing constants, replacing operators, modifying branch conditions, and deleting statements. Studies

have shown that, given two test suites for the same system under test, the one that detects more mutants (even using only these limited mutation operators) is likely to also detect more real faults [5], [6].

However, not *all* real faults are coupled to mutants due to the limited set of mutation operators. For example, a fault resulting from calling the wrong method on an object is unlikely to be coupled to a mutant, as state-of-the-art mutation tools do not implement a “change method call” operator. While a far wider range of mutation operators has been explored in the literature [7], [8], state-of-the-practice tools like Pitest [9], [10], Major [11], and Stryker [12] typically do not implement them because of the implementation effort required and, especially, the increased cost of mutation analysis. Each additional mutation operator will result in more mutants that must be run and analyzed. Since each mutant must be evaluated in isolation, this may dramatically increase the time needed for developers that use the tool. Furthermore, some mutation operators might not be worthwhile to run, as noted in documentation from the developer of Pitest: “Although pitest provides a number of other operators, they are not enabled by default as they may provide a poorer experience” [13]. An alternative approach for generating mutants is to use a dataset of real faults to train a machine learning model to learn how to inject mutants [14], [15], [16]. However, the need for developers to train a model for their project impedes adoption of such techniques.

Our approach, *LLMorpheus*, can be viewed as a generalization of rule-based mutation techniques [9], [10], [11], [12] in which the location of mutations is determined using a set of predefined rules and where an LLM is asked to suggest a diversity of mutations that introduce buggy behavior at those locations. To this end, *LLMorpheus* repeatedly prompts an LLM to inject faults at designated locations into a code fragment using prompts that include: (i) general background on mutation testing (ii) (parts of) a source file in which a single code fragment is replaced with the word “PLACEHOLDER”, (iii) the original code fragment that was replaced by the placeholder, and (iv) a request to replace the placeholder with a buggy code fragment that has different behavior than the original code. After discarding syntactically invalid suggestions, we use *StrykerJS*, a state-of-the-art mutation testing tool for JavaScript that we modified to apply the mutations suggested by *LLMorpheus* instead of applying its standard mutators, classify mutants as killed, surviving, or timed out, and generate an interactive web site for inspecting the results.

Received 26 July 2024; revised 27 March 2025; accepted 12 April 2025. Date of publication 25 April 2025; date of current version 16 June 2025. This work was supported in part by US National Science Foundation under Grant CCF-2100037, Grant CCF-2307742, and Grant CNS-2100015. Recommended for acceptance by M. Sabetzadeh. (Corresponding author: Frank Tip.)

Frank Tip and Jonathan Bell are with the Northeastern University, Boston, MA 02115 USA (e-mail: f.tip@northeastern.edu).

Max Schäfer is with the XBOW, 1920 Tas-Sliema, Malta.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSE.2025.3562025>, provided by the authors.

Digital Object Identifier 10.1109/TSE.2025.3562025

The effectiveness of our approach hinges on the assumption that LLMs can understand the surrounding context of the code fragment represented by a PLACEHOLDER well enough to suggest syntactically valid and realistic buggy code fragments. To determine whether this assumption holds, we evaluate *LLMorpheus* on 13 subject applications written in JavaScript and TypeScript and measure how many mutants are generated and how they are classified (killed, survived, timed-out) using four “open” LLMs for which the training process is documented (Meta’s *codellama-34b-instruct*, *codellama-13b-instruct*, *llama-3.3-70b-instruct* and Mistral’s *mixtral-8x7b-instruct*) and one proprietary LLM (OpenAI’s *gpt-4o-mini*). We manually examine a subset of the surviving mutants to determine whether they are equivalent to the original source code or if they represent behavioral changes and contrast the results against mutants generated using *StrykerJS*’s standard mutators. The cost of *LLMorpheus* is assessed by measuring its running time and the number of tokens used for prompts and completions. We also report on experiments with alternative prompts that omit parts of the information encoded in default prompts and with different “temperature” settings of an LLM.

For surviving mutants generated using *codellama-34b-instruct*, we find that the majority (80%) reflect behavioral differences and 20% are equivalent to the original code. Using the *codellama-34b-instruct* and *codellama-13b-instruct* models, results are generally stable at temperature 0.0 when experiments are repeated, but the use of higher temperatures yields more variable results. For *mixtral-8x7b-instruct*, *llama-3.3-70b-instruct*, and *gpt-4o-mini* models, there is already significant variability at temperature 0. The default template generally produces the largest number of mutants and surviving mutants, and removing different fragments of this prompt degrades the results to varying degrees. The *llama-3.3-70b-instruct* and *codellama-34b-instruct* LLMs generally produce the largest number of mutants and surviving mutants, but *LLMorpheus* is still effective when *codellama-13b-instruct*, *mixtral-8x7b-instruct*, and *gpt-4o-mini* are used.

To investigate *LLMorpheus*’s ability to produce mutants that resemble real-world faults, we conducted a detailed case study involving 40 real-world bugs. In this case study, we used *LLMorpheus* to mutate the *fixed* version of a program near the location of the fix, executed the program’s tests for each of these mutants and compared the test outcomes against those of the buggy version. For the 40 bugs under consideration in the case study, *LLMorpheus* was able to produce mutants that are syntactically identical to the buggy code fragments in 10 cases, and mutants that produce the same test failures as the original bug in an additional 26 cases. This provides evidence that *LLMorpheus* is capable of generating mutants whose behavior resembles that of real-world bugs, and that this capability is not entirely due to training-set leakage.

In summary, the contributions of this paper are:

- 1) A technique for mutation testing in which placeholders are introduced at designated locations in a program’s source code, and where an LLM is prompted to suggest what they could be replaced with.

(a)

```

118 118         }
119 119         try {
120 -         await fs.promises.access(srcFolder, fs.constants.R_OK | fs.constants.W_OK);
120 +         await fs.promises.access(srcFolder, fs.constants.R_OK); //eslint-disable-line no-b
121 121         await fs.promises.access(targetBasePath, fs.constants.R_OK | fs.constants.W_OK);

```

(b)

```

120         try {
121         if (!isGlob(src)) { *****
122 -         await fs.promises.access(src, fs.constants.R_OK); //eslint-disable-line no-b
122 +         await fs.promises.access(src, fs.constants.W_OK); //eslint-disable-line no-b
123         }
124         await fs.promises.access(targetBasePath, fs.constants.R_OK | fs.constants.W_OK);

```

(c)

```

120         try {
121         if (!isGlob(src)) { *****
122         await fs.promises.access(src, fs.constants.R_OK); //eslint-disable-line no-b
123         }
124 -         await fs.promises.access(targetBasePath, fs.constants.R_OK | fs.constants.W_OK);
124 +         await fs.promises.access(targetBasePath, fs.constants.R_OK); //eslint-disable-line no-b
125         } catch (e: any) {

```

Fig. 1. (a) Fix for a bug reported in issue #36 in zip-a-folder. (b) A mutation suggested by *LLMorpheus* at the same line that involves replacing read-access with write-access. (c) A mutation suggested by *LLMorpheus* elsewhere in the same file that mirrors the change made by the developer.

- 2) An implementation of this technique in *LLMorpheus*, a practical mutation testing tool for JavaScript.
- 3) An empirical evaluation of *LLMorpheus* on 13 subject applications, demonstrating its practicality and comparing it to a standard approach to mutation testing based on mutation operators.
- 4) A case study demonstrating *LLMorpheus*’ ability to produce mutants with behavior resembling that of real-world bugs.

The remainder of this paper is organized as follows. Section II presents motivating examples that illustrate the potential of LLM-based mutation techniques to introduce faults resembling real bugs. In Section III, an overview of our approach is presented. Section IV presents an evaluation of *LLMorpheus* and Section V covers threats to validity. Related work is discussed in Section VI. Lastly, Section VII presents conclusions and directions for future work.

II. BACKGROUND AND MOTIVATION

In this section, we study a few bugs that do not correspond to mutation operators supported by state-of-the-art mutation testing tools but that are similar to mutations *LLMorpheus* could suggest.

a) Example 1: Zip-a-folder [17] is a library for compressing folders. On January 31, 2022, a user observed that the library required write access for source folders unnecessarily and opened issue #36, requesting that this access be removed. The developer applied the fix shown in Fig. 1(a) on the same day, which involves replacing a binary bitwise-or expression with one of its operands.

LLMorpheus can suggest mutations that involve *changing* or *introducing* references to functions, variables, and properties. Fig. 1(b) and 1(c) show two mutations that *LLMorpheus* suggests for this project and that could result in bugs similar to the one described above: part (b) shows a mutation at the same line where the bug was located that involves replacing read access

Figure 2(a) shows a code diff for the `getOffsetStr` function. Line 30 has a minus sign and line 31 has a plus sign, indicating a change from `Math.floor(offset / 60)` to `Math.floor(Math.abs(offset) / 60)`. Figure 2(b) shows the original code for `getOffsetStr` and `getNumStr`. In `getNumStr`, line 38 has a minus sign and line 39 has a plus sign, indicating a change from `Math.abs(input)` to `Math.round(input)`.

Fig. 2. (a) Fix for a bug reported in issue #60 in countries-and-timezones. (b) A mutation suggested by *LLMorpheus* elsewhere in the same file.

with write access and part (c) shows a mutation at a nearby location that mirrors the change made by the developer.

The state-of-the-art *StrykerJS* tool is unable to suggest either of these mutations because (i) it does not support the mutation of bitwise operator expressions such as `fs.constants.R_OK | fs.constants.W_OK` unless they appear as part of a control-flow predicate, nor (ii) mutations that involve replacing a binary expression with one of its operands. While adding support for mutating bitwise operator expressions would be straightforward, concerns have been expressed that adding more mutation operators to traditional mutation testing tools might result in too many mutants and degraded performance [13], [18], [19]. More significantly, *StrykerJS* does not introduce or modify property access expressions and has very limited support for replacing an expression with a different expression¹.

b) Example 2: Countries-and-timezones [20] is a library for working with countries and timezones. In October 2023, a user reported a bug in function `getOffsetStr`, stating that it produces incorrect results when invoked with negative values. The developer proposed a simple fix that involves inserting a call to `Math.abs` to convert the argument value to a non-negative number, and a variation on this fix was quickly adopted by the developer, as shown in Fig. 2(a).

This bug fix involves the introduction of a function call, so to *introduce* bugs like this one, a mutation testing tool would have to remove function calls or change the function being invoked. *StrykerJS* only supports a very limited set of 20 mutations to function calls², such as replacing calls to `String.startsWith` with call to `String.endsWith` and removing a call to `Array.slice`. While one could extend *StrykerJS* with a mutator that removes calls to `Math.abs`, many other function calls could be handled similarly, and adding mutators for all of them would yield an overwhelmingly large number of mutants. Many such candidate functions would not be good choices for mutation,

¹In particular, *StrykerJS* only replaces control-flow predicates in `if`-statements and loops with boolean constants, string literals with the value “Stryker was here”, and object literals with an empty object literal.

²See <https://stryker-mutator.io/docs/mutation-testing-elements/supported-mutators/>.

Figure 3(a) shows a code diff for the `image-downloader` module. Line 22 has a minus sign and line 23 has a plus sign, indicating a change from `path.resolve` to `path.join`. Figure 3(b) shows the original code for the same module. In line 22, there is a plus sign and in line 23, there is a minus sign, indicating a change from `path.join` to `path.basename`.

Fig. 3. (a) Fix for a bug reported in issue #27 in image-downloader. (b) A mutation suggested by *LLMorpheus* at the same location that similarly involves calling a different function.

Figure 4 shows a code diff for a `Promise` function. Line 138 has a minus sign and line 139 has a plus sign, indicating a change from `output.on('close', resolve)` to `output.on('end', resolve)`.

Fig. 4. A mutation suggested by *LLMorpheus* that involves associating an event listener with the end event instead of with the close event.

either because the function in question is not a function that a developer inadvertently might have selected or because it would lead to syntactically invalid code.

LLMorpheus suggests mutations that involve introducing and replacing function calls. Fig. 2(b) shows a mutation that *LLMorpheus* suggested elsewhere in the same source file that replaces a call to `Math.abs` with a call to `Math.round`, which could, in principle, introduce a bug like the one in Fig. 2(a). Moreover, since LLMs are trained to generate code that resembles code written by developers, it is likely that the mutants produced by *LLMorpheus* involve using functions that a developer might have chosen.

c) Example 3: image-downloader is a module for downloading images. In February 2022, a user opened issue #27, entitled “If the directory name in dest: contains a dot “.” then the download fails.”, providing an example illustrating the problem. The developers soon responded with a fix, shown in Fig. 3(a), that involves replacing a call to `path.resolve` with a call to `path.join`. While *LLMorpheus* does not produce a mutant that re-introduces this bug exactly, it does produce several at the same location³ that similarly replace the invoked function, including the one shown in Fig. 3(b). As mentioned, *StrykerJS* has very limited support for mutations that involve calling different functions and so it cannot suggest mutations like the one shown in Fig. 3(b).

³The line numbers have shifted slightly as the code has evolved since the bug report.



Fig. 5. Overview of approach.

d) *Example 4*: Fig. 4 shows another mutant produced by *LLMorpheus* for *zip-a-folder*. Here, the mutation involves changing the name of the event with which an event listener is associated. Such errors often cause “dead listeners”, i.e., situations where an event handler is never executed because it is associated with the wrong event. Dead listeners are quite common in JavaScript, where the use of string values to identify events precludes static checking, and previous research has focused on static analysis [21] and statistical methods [22] for detecting such errors.

e) *Discussion*: The above examples illustrate just a few of the kinds of mutations that *LLMorpheus* may produce. Other mutations that it may suggest include: replacing a reference to a variable with a reference to a different variable, adding or removing arguments in function calls, and modifying object literals by adding or removing property-value pairs.

In practice, the number of such mutations is effectively infinite, so an approach based on exhaustively applying a fixed set of mutation operators is unlikely to be practical. *LLMorpheus*’ LLM-based approach leverages the collective wisdom of programmers who wrote the code on which the LLM was trained to develop mutations. As a result, suggested changes are likely to refer only to variables and functions that are in scope and are likely to be type-correct.

III. APPROACH

LLMorpheus is capable of producing interesting mutants without requiring any training on a subject project, which is a key distinction compared to existing work that builds models of real bugs to generate mutants [14], [15], [16]. This is accomplished by querying an LLM with a prompt that includes part of an application’s source code in which a code fragment is replaced with the text “<PLACEHOLDER>”. Additional information provided in the prompt includes: (i) general background on mutation testing, (ii) the code fragment that was originally present at the placeholder’s location, (iii) a request to apply mutation testing to the code by replacing the placeholder with a buggy code fragment, and (iv) suggestions *how* the code could be mutated. The LLM is asked to provide three possible replacements for the placeholder⁴, each accompanied by an explanation how the mutation would change program behavior.

⁴The mutants produced by *LLMorpheus* always contain exactly one code change; if three valid suggestions are received from the LLM in response to one prompt, then three separate mutants will be generated.

<code>if (x === y){ ... }</code>	<code>if (<PLACEHOLDER>){ ... }</code>
<code>switch (x === y){ ... }</code>	<code>switch (<PLACEHOLDER>){ ... }</code>
<code>while (x){ ... }</code>	<code>while (<PLACEHOLDER>){ ... }</code>
<code>do { ... } while (x);</code>	<code>do { ... } while (<PLACEHOLDER>);</code>
<code>for (let i=0; i < x; i++){ ... }</code>	<code>for (<PLACEHOLDER>; i < x; i++){ ... }</code>
<code>for (let i=0; i < x; i++){ ... }</code>	<code>for (let i=0; i < <PLACEHOLDER>; i++){ ... }</code>
<code>for (let i=0; i < x; i++){ ... }</code>	<code>for (<PLACEHOLDER>; i < x; i++){ ... }</code>
<code>for (o in obj){ ... }</code>	<code>for (<PLACEHOLDER> in obj){ ... }</code>
<code>for (o in obj){ ... }</code>	<code>for (o in <PLACEHOLDER>){ ... }</code>
<code>for (o of obj){ ... }</code>	<code>for (<PLACEHOLDER> of obj){ ... }</code>
<code>for (o of obj){ ... }</code>	<code>for (o of <PLACEHOLDER>){ ... }</code>
<code>a.m(x,y)</code>	<code><PLACEHOLDER>(x,y)</code> <code>a.m(<PLACEHOLDER>,y)</code> <code>a.m(x,<PLACEHOLDER>)</code> <code>a.m(<PLACEHOLDER>)</code>

Fig. 6. Illustration of the insertion of placeholders to direct the LLM at source locations that need to be mutated.

Fig. 5 presents a high-level overview of our approach, which involves three components that work in concert: the *prompt generator*, the *mutant generator*, and a version of the *StrykerJS* mutation testing tool that has been modified to apply the mutants created by *LLMorpheus*⁵. We now discuss each of these components.

a) *Prompt Generator*: This component takes as input a package and generates a set of prompts. This involves parsing the source files and identifying locations where mutations will be introduced. For ease of reference during prompting, the source code fragment corresponding to each location is replaced with the text “<PLACEHOLDER>”. *LLMorpheus* considers the following locations as candidates for mutation: (i) conditions of **if**, **switch**, **while**, and **do-while** statements, (ii) initializers, updaters, and entire headers of loop statements, and (iii) receiver, arguments, and entire sequence of arguments for function calls. For each such location, a separate prompt is created. Fig. 6 illustrates where placeholders are introduced into the source code.

The LLM is then given a prompt that is created by instantiating the template shown in Fig. 7(a), by replacing `{{code}}` with the original source code in which a placeholder has been inserted, and `{{orig}}` with the code fragment that was replaced by the placeholder. Fig. 7(b) shows the system prompt given to the LLM, which provides background on the role the LLM is expected to play in the conversation as a mutation testing expert. As can be seen in Fig. 7(a), the prompt provides

⁵In particular, we use *StrykerJS* to (i) determine the impact of each mutant on an application’s tests and classify it as “killed”, “survived”, or “timed-out” and (ii) generate an interactive web page for inspecting results.

```

Your task is to apply mutation testing to the following
code:
'''
{{{code}}}
'''

by replacing the PLACEHOLDER with a buggy code fragment
that has different behavior than the original code fragment,
which was:
'''
{{{orig}}}
'''

Please consider changes such as using different operators,
changing constants, referring to different variables, object
properties, functions, or methods.

Provide three answers as fenced code blocks containing a
single line of code, using the following template:

Option 1: The PLACEHOLDER can be replaced with:
'''
<code fragment>
'''
This would result in different behavior because
<brief explanation>.

Option 2: The PLACEHOLDER can be replaced with:
'''
<code fragment>
'''
This would result in different behavior because
<brief explanation>.

Option 3: The PLACEHOLDER can be replaced with:
'''
<code fragment>
'''
This would result in different behavior because
<brief explanation>.

Please conclude your response with "DONE."
(a)

You are an expert in mutation testing. Your job is to
make small changes to a project's code in order to find
weaknesses in its test suite. If none of the tests fail
after you make a change, that indicates that the tests
may not be as effective as the developers might have
hoped, and provide them with a starting point for
improving their test suite.
(b)

```

Fig. 7. Prompt template: (a) and system prompt; (b) used by *LLMorpheus*.

instructions for applying mutation testing to the specific source code at hand and details the format to which the completion should conform. Specifically, we require that the proposed mutants be provided inside “fenced code blocks” (i.e., code blocks surrounded by three backquote characters).

b) Mutant Generator: This component takes the completions received from the LLM and extracts candidate mutants from the instantiated template by matching a regular expression against the completion to find the fenced code blocks. Candidate mutants identical to the original source code fragment or identical to previously generated mutants are discarded. The candidate mutants are then parsed to check if they are syntactically valid and discarded if this is not the case. The resulting mutants are written to a file `mutants.json` that is read by a customized version of *StrykerJS* that is described below. The mutant generator also saves all experimental data to files, including the generated prompts, completions received from the LLM, and the configuration options (e.g., the LLM’s temperature setting).

c) Custom Version of StrykerJS: We modified *StrykerJS* to give it an option `--usePrecomputed` that, if selected, directs it to read its set of mutations from a file `mutants.json` instead. *StrykerJS* then executes all mutants and determines (for each mutant) whether it causes tests to fail or time out. When this analysis is complete, *StrykerJS* generates a report as an interactive web page, allowing users to inspect the generated mutants. The previously shown Figs. 1–4 show screenshots of our custom version of *StrykerJS*.

d) Pragmatics: While *LLMorpheus* implements a conceptually straightforward technique, considerable engineering effort was required to make it a practical tool. We use *BabelJS* [23] for parsing source code to identify locations where placeholders should be inserted and to check the syntactic validity of candidate mutants. *Handlebars* [24] is used for instantiating prompt templates. *StrykerJS* expects mutants to correspond to a single AST node, so for mutants that do not correspond exactly to a single AST node (e.g., loop headers and sequences of arguments passed in function calls), it is necessary to expand the mutation to the nearest enclosing AST node, for which we also rely on *BabelJS*.

LLMorpheus has command-line arguments for specifying the prompt template and system template to be used. Furthermore, it enables users to specify a number of LLM-specific parameters, such as the maximum length of completions that should be generated, the sampling temperature⁶, and number of lines of source code that should be included in prompts (by default, this is limited to 200 lines surrounding the location of the placeholder). Since many LLM installations have limited capacity or explicit rate limits, *LLMorpheus* provides two command-line options to work with such LLMs: `--rateLimit <N>` ensures that at least *N* milliseconds will have elapsed between successive prompts and `--nrAttempts <N>` will try the same prompt up to *N* times if a 429 error occurs.

One possible concern with our approach is that *LLMorpheus* relies on a fixed set of locations where it introduces placeholders. The current placeholder scheme aims to balance creating a practical number of mutants and a larger set of mutants where at least one is more likely to result in a different control flow or data flow. Modifying *LLMorpheus* to use a different placeholder scheme would be straightforward. That said, the examples in Section II show that mutants produced by *LLMorpheus* (using its current placeholder scheme) involve changing references to variables, properties, and functions that cannot be produced using Stryker’s mutation operators and that correspond to real-world bugs.

An open-source release of *LLMorpheus* can be found at <https://github.com/neu-se/llmorpheus> and the customized

⁶The sampling temperature is a parameter between 0 and 2 that controls the randomness of the completions generated by the LLM. Roughly speaking, the higher the temperature the more diverse the completions. At temperature zero, the LLM will always generate the most likely completion, which increases the chance that the same prompt will result in the same completion.

TABLE I
SUBJECT APPLICATIONS USED TO EVALUATE *LLMORPHEUS*

Application	Description	Weekly Downloads	#LOC	#Tests	Coverage		StrykerJS					
					stmt	branch	#mutants	#killed	#survived	#timeout	mut. score	time (sec)
<i>Complex.js</i>	complex numbers	671K	1,425	216	71.82%	67.54%	1,302	763	539	0	58.60	405.08
<i>countries-and-timezones</i>	accessing countries and timezones data	152K	165	58	100%	92.55%	140	134	6	0	95.71	142.37
<i>crawler-url-parser</i>	URL parser for crawling	495	209	185	96.39%	92.5%	226	143	83	0	63.27	433.53
<i>delta</i>	format for representing rich text documents and changes	1.76M	806	180	98.99%	95.89%	834	686	88	60	89.45	2747.04
<i>image-downloader</i>	downloading image to disk from a given URL	17.75K	64	11	100%	93.75%	43	28	11	4	74.42	284.20
<i>node-dirty</i>	key value store with append-only disk log	5,604	207	37	83.01%	71.15%	160	78	56	26	65.00	215.93
<i>node-geo-point</i>	calculations involving geographical coordinates	5,618	406	10	85.36%	70.58%	158	98	60	0	62.03	357.70
<i>node-jsonfile</i>	reading/writing JSON files	57.7M	102	43	97.87%	94.11%	61	31	5	25	91.80	188.91
<i>plural</i>	plural forms of nouns	2,271	103	14	95.38%	72.72%	180	143	37	0	79.44	53.66
<i>pull-stream</i>	pipeable pull-stream	57.8K	602	364	90.96%	80.84%	474	318	116	40	75.53	694.33
<i>q</i>	promises	10.1M	2,111	243	89.5%	70.92%	1,058	68	927	63	12.38	7,075.02
<i>spacl-core</i>	path-based access control	3	377	38	100%	100%	259	239	20	0	92.28	1,053.16
<i>zip-a-folder</i>	zip/tar utility	60.1K	156	22	100%	96.87%	74	38	8	28	89.19	513.27

version of *StrykerJS* that we used for classifying mutants can be found at <https://github.com/neu-se/stryker-js>.

IV. EVALUATION

A. Research Questions

This evaluation aims to answer the following research questions:

- RQ1** How many mutants does *LLMorpheus* create?
- RQ2** How many of the surviving mutants produced by *LLMorpheus* are equivalent mutants?
- RQ3** What is the effect of using different temperature settings?
- RQ4** What is the effect of variations in the prompting strategy used by *LLMorpheus*?
- RQ5** How does the effectiveness of *LLMorpheus* depend on the LLM that is being used?
- RQ6** What is the cost of running *LLMorpheus*?
- RQ7** Is *LLMorpheus* capable of producing mutants that resemble existing bugs?

B. Experimental Setup

a) Selecting Subject Applications: Our goal is to evaluate *LLMorpheus* on real-world JavaScript packages that have test suites. Moreover, we want to compare the mutants generated by *LLMorpheus* to those generated using traditional mutation testing techniques, so we decided to focus on projects for which the state-of-the-art StrykerJS mutation testing tool [12] could be applied successfully. As a starting point for benchmark selection, we considered the 25 subject applications that were used to evaluate TestPilot [25], a recent LLM-based unit test generation tool. These applications are written in JavaScript or TypeScript, cover various domains, and have test suites that can be executed successfully.

Of these 25 subject applications, 10 could not be used because StrykerJS does not work on them, either because its dependences conflict with those of the subject application

itself⁷, or because it crashes. On one package, *simple-statistics*, StrykerJS requires approximately 10 hours of running time, which makes using it impractical. We excluded another package, *fs-extra*, a utility library for accessing the file system, because we observed that mutating this application poses a significant security risk, as the mutated code was corrupting our local file system. This left us with 13 subject applications for which Table I provides key characteristics. The first set of columns in the table show, from left to right, the name of the package, a short description of its functionality, the number of weekly downloads according to npmjs.com, the number of lines of source code, the number of tests, and the statement and branch coverage achieved by those tests, respectively. The second set of columns shows the results of running *StrykerJS* on the applications: the total set of mutants, the number of mutants that were killed, survived, and that timed out, the *mutation score*⁸ reported by *StrykerJS*, and the time required to run *StrykerJS*, respectively.

b) LLM Selection: RQ5 explores how the effectiveness of the proposed technique depends on the LLM being used. We use Meta's *codellama-34b-instruct* model for our main experiments. In addition, we evaluate the technique with Meta's *codellama-13b-instruct* and *llama-3.3-70b-instruct* models, with Mistral's *mixtral-8x7b-instruct* model, and with OpenAI's *gpt-4o-mini* model. The *codellama* models are specifically trained for tasks involving code. *llama-3.3-70b-instruct* is a newer and larger model from Meta that supersedes the smaller, specialized *codellama* models. *mixtral-8x7b-instruct* is a state-of-the-art general-purpose "mixture-of-experts" LLM developed by Mistral. *gpt-4o-mini* is a smaller, faster, and lower-cost

⁷Running StrykerJS on an application requires installing it locally among the subject project libraries. Stryker itself depends on various other packages that also need to be installed, and these packages may conflict with packages that the subject application itself depends upon.

⁸The mutation score aims to provide a measure of the quality of a test suite by calculating the fraction of the total number of mutants that are detected (i.e., killed or timed out), see <https://stryker-mutator.io/docs/General/faq/>.

TABLE II

RESULTS FROM LLMORPHEUS EXPERIMENT (RUN #312). MODEL: *CODELLAMA-34B-INSTRUCT*, TEMPERATURE: 0.0, MAXTOKENS: 250, TEMPLATE: *TEMPLATE-FULL.HB*, SYSTEMPROMPT: *SYSTEMPROMPT-MUTATIONTESTINGEXPERT.TXT*

Application	#Prompts	LLMorpheus									StrykerJS				
		#candidates	#invalid	#identical	#duplicate	#mutants	#killed	#survived	#timeout	mut. score	#mutants	#killed	#survived	#timeout	mut. score
Complex.js	490	1,451	194	13	45	1,199	725	473	1	60.55	1,302	763	539	0	58.60
countries-and-timezones	106	318	89	0	12	217	188	29	0	86.64	140	134	6	0	95.71
crawler-url-parser	176	521	205	14	17	285	157	128	0	55.09	226	143	83	0	63.27
delta	462	1,367	565	10	25	767	634	101	32	86.83	834	686	88	60	89.45
image-downloader	42	124	33	2	0	89	72	17	0	80.90	43	28	11	4	74.42
node-dirty	154	450	153	15	7	275	163	100	12	63.64	160	78	56	26	65.00
node-geo-point	140	408	93	0	13	302	223	79	0	73.84	158	98	60	0	62.03
node-jsonfile	68	199	42	3	0	154	49	48	57	68.83	61	31	5	25	91.80
plural	153	442	101	42	18	281	205	75	1	73.31	180	143	37	0	79.44
pull-stream	351	1,028	238	12	9	769	441	271	57	64.76	474	318	116	40	75.53
q	1,051	3,121	1,000	34	52	2,035	158	1,792	85	11.94	1,058	68	927	63	12.38
spacel-core	134	395	140	10	6	239	199	39	1	83.68	259	239	20	0	92.28
zip-a-folder	49	143	41	1	1	100	23	3	74	97.00	74	38	8	28	89.19
Total	3,376	9,967	2,894	156	205	6,712	3,237	3,155	320	—	4,969	2,767	1,956	246	—

variant of OpenAI’s popular *gpt-4o* model The *codellama-34b-instruct*, *codellama-13b-instruct*, *llama-3.3-70b-instruct*, and *mixtral-8x7b-instruct* LLMs are “open” in the sense that their training process is documented. We relied on several commercial LLM service providers (<https://octo.ai>, <https://openai.com>, and <https://openrouter.ai>) for the experiments reported on in this paper.

c) *LLM Temperature Settings*: LLMs have a temperature parameter that reflects the amount of randomness or creativity in their completions. For a task such as mutation testing, randomness and creativity may determine whether generated mutants are killed or survive. Therefore, we conduct experiments using several temperature settings.

d) *Similarity to Real-World Bugs*: Previous work evaluating mutation testing techniques has focused on “coupling” to determine whether mutants resemble real-world bugs [5], [6], [26]. This involves determining whether a test suite that detects particular mutants also detects particular real faults and requires a curated dataset of isolated faults. While many such datasets have been constructed from open-source projects written in Java, we found only one JavaScript dataset, the Bugs.js suite [27]. For each of these bugs, the original faulty version is provided, along with a cleaned patch extracted from the bug fix and instructions on executing the test cases. Unfortunately, we found that most of the Bugs.js subjects could not be used at all due to their reliance on outdated versions of various libraries and because of their incompatibility with modern Node.js versions that *StrykerJS* requires, causing them to be incompatible with *LLMorpheus*. These projects also have flaky tests⁹, making it particularly challenging to perform mutation analysis [28].

We therefore constructed a new dataset¹⁰ consisting of 40 real-world bugs, which includes 4 real-world bugs from the

Bugs.js suite that we could reproduce reliably and 36 real-world bugs that we manually curated from various Node.js applications that are available from GitHub (including some of the applications listed in Table I).

We identified these bugs by searching <https://www.npmjs.com/> for Node.js applications that cover a variety of domains, studying the issues in the GitHub repositories associated with these applications for messages that indicated the presence of a bug, and finding a subsequent “bug fix” commit in which this bug had been fixed (in most cases these also included the addition of new tests, or changes to existing tests). We then cloned the repository at that commit, reintroduced the bug, and observed if it caused any test failures. We discarded projects in which bugs/issues are not tracked explicitly and we discarded bugs that did not cause any test failures when reintroduced. In addition, we restricted our attention to bugs for which the commit containing the fix involves changing at most three lines of source code. Section IV-I will report on experiments in which mutants are introduced at these locations, and our goal was to keep the number of such mutants manageable, given that careful manual analysis is involved in comparing the behavior of each mutant to that of the original buggy code.

C. RQ1: How Many Mutants Does LLMorpheus Create?

To answer this question, we ran *LLMorpheus* on the projects listed in Table I using the *codellama-34b-instruct* LLM at temperature 0.0 and the prompt templates shown in Fig. 7. The results, shown in Table II, show that *LLMorpheus* produces between 42 and 1,051 prompts for these projects. The following four columns in the table show the number of “candidate mutants”, i.e., code fragments obtained by replacing placeholders with code fragments suggested by the LLM. The subcolumn labeled “candidates” shows the total number of candidate mutants, the subcolumn labeled “invalid” shows the number of candidate mutants that were found to be syntactically invalid, the subcolumn labeled “identical” shows the number of candidate mutants that were found to be identical to the original code,

⁹See <https://github.com/BugsJS/bug-dataset/issues/11>.

¹⁰To facilitate further research by the community, our new bug dataset is available from <https://github.com/neu-se/mutation-testing-data> along with all experimental results associated with this paper.

TABLE III
NUMBER OF MUTANTS GENERATED USING THE *CODELLAMA-34B-INSTRUCT* LLM AT TEMPERATURES 0.0, 0.25, 0.5, AND 1.0
(SHOWING ONE RUN OF EACH)

	temp. 0.0 (run #312)				temp. 0.25 (run #348)				temp. 0.50 (run #318)				temp. 1.0 (run #341)			
	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout
<i>Complex.js</i>	1,199	725	473	1	1,197	730	466	1	1,191	739	452	0	1,028	648	379	1
<i>countries-and-timezones</i>	217	188	29	0	219	181	38	0	224	194	30	0	186	156	30	0
<i>crawler-url-parser</i>	285	157	128	0	260	167	93	0	298	166	108	24	278	202	76	0
<i>delta</i>	767	634	101	32	781	642	111	28	769	642	93	34	698	583	83	32
<i>image-downloader</i>	89	72	17	0	86	71	15	0	89	68	21	0	75	53	22	0
<i>node-dirty</i>	275	163	100	12	279	175	93	11	277	158	107	12	246	150	84	12
<i>node-geo-point</i>	302	223	79	0	293	225	68	0	302	230	72	0	273	213	60	0
<i>node-jsonfile</i>	154	49	48	57	151	52	41	58	153	51	43	59	132	50	22	60
<i>plural</i>	281	205	75	1	273	208	63	2	289	219	69	1	299	229	69	1
<i>pull-stream</i>	769	441	271	57	779	452	270	57	796	465	278	53	743	461	235	47
<i>q</i>	2,035	158	1,792	85	2,050	153	1,813	84	2,073	163	1,823	87	1,899	147	1,671	81
<i>spacel-core</i>	239	199	39	1	223	187	36	0	250	210	39	1	218	180	38	0
<i>zip-a-folder</i>	100	23	3	74	97	24	4	69	87	48	33	6	96	54	38	4
<i>Total</i>	6,712	3,237	3,155	320	6,688	3,267	3,111	310	6,798	3,353	3,168	277	6,171	3,126	2,807	238

and the subcolumn labeled “duplicate” shows the number of candidate mutants that were found to be duplicated. From this data, it can be inferred that, on average, 29.0% (2,894/9,967) of candidate mutants are discarded because they are syntactically invalid, 1.6% (156/9,967) are discarded because they are identical to the original code, and 2.1% (205/9,967) are discarded because they are duplicates. This suggests that LLMs generally do not have too much trouble with generating syntactically correct code, which is consistent with recent findings by others [25], [29].

The next column, labeled “mutants”, shows the number of remaining mutants after discarding the useless candidate mutants. Here, the reader can see that between 89 and 2,035 mutants are generated for the subject packages. Of these mutants, between 23 and 725 are killed, between 3 and 1,792 survive, and between 0 and 85 time out. Aggregating the results over the 13 projects, it can be seen that 48.2% (3,237/6,712) of all mutants are killed, 47.0% (3,155/6,712) of all mutants survive, and 4.8% (320/6,712) of all mutants time out.

The table also shows the *mutation score*¹¹ as reported by StrykerJS, which aims to provide a measure of the quality of a test suite by calculating the fraction of the total number of detected mutants (i.e., killed or timed out).

To facilitate a quantitative comparison with *StrykerJS*, the last five columns in Table II repeat the results of running *StrykerJS* on the subject applications from Table I. From this data, it can be seen that—in the aggregate for the 13 projects under consideration—*LLMorpheus* produces 3,155 surviving mutants whereas *StrykerJS* produces 1,956 surviving mutants. However, it should be noted that the difference in the number of mutants and surviving mutants varies significantly between subject applications. For example, for *Complex.js* *StrykerJS* produces more mutants (1,302 vs. 1,199) than *LLMorpheus*, of which more survive (539 vs. 473). On the other hand, for *q*, the situation is reversed with *StrykerJS* producing fewer mutants (1,058 vs. 2,035) and fewer surviving mutants (927 vs. 1,792) than

LLMorpheus. We conjecture that such differences are due to the subject programs’ different characteristics, which make them amenable to different types of mutations. Here, *Complex.js* heavily uses arithmetic operators to implement mathematical operations on complex numbers, and such operators are prime candidates for *StrykerJS*’s standard mutation operators. Moreover, *q* makes heavy use of method calls, which are targeted by *LLMorpheus*’s placeholder-based strategy but much less so by *StrykerJS*’s mutation operators.

LLMs are nondeterministic, even at temperature 0.0, so a subsequent experiment may produce results that differ from those shown in Table II. To determine to what extent this is the case, we repeated the same experiment four more times and measured how often the same mutants occur. We found that, at temperature 0.0, the results of *LLMorpheus* are generally stable across runs, with between 89.29% and 98.89% of all mutants being observed in all 5 experiments¹². Fig. 8 visualizes the stability of *LLMorpheus* when varying the prompt settings, and our supplemental materials include results across all settings of all models.

Using *codellama-34b-instruct* at temperature 0.0, *LLMorpheus* generates between 89 and 2,035 mutants, of which between 3 and 1,792 survive. These results are stable across experiments, with between 89.29% and 98.89% of all mutants being observed in all five experiments.

D. RQ2: How Many of the Surviving Mutants Are Equivalent Mutants?

One of the key challenges in mutation testing is the phenomenon of *equivalent mutants*: mutants that have equivalent behavior as the original code [4]. Mutants produced by *LLMorpheus* may involve arbitrary code changes, so the LLM could suggest code that is effectively a refactored version of the code that was originally present. To determine to what extent surviving mutants produced by *LLMorpheus* are equivalent, we

¹¹See <https://stryker-mutator.io/docs/General/faq/>.

¹²All experimental data associated with this experiment and the other experiments are included with this submission as supplemental materials.

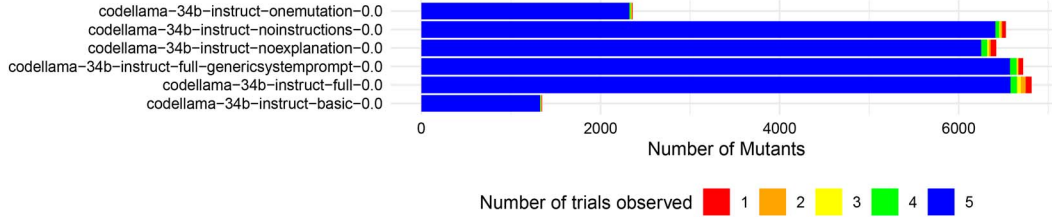


Fig. 8. Stability of mutants generated by LLMorpheus with *codellama-34b-instruct* at temperature 0.0. For each replacement generated at each position, we count the number of trials (of 5 total) where that replacement was generated.

conducted a study in which two authors manually examined 50 surviving mutants¹³ in each project and classified each mutant as “equivalent” or “not equivalent” by Stryker or by *LLMorpheus* (sampled from run 314).

We labeled a mutant as *equivalent* if we could determine that the change could not cause *any* observable difference in behavior. For example, mutants that added extra parameters to methods (beyond those accepted by the receiver method) are trivially equivalent, as the runtime discards them. Other mutants are far from trivial to evaluate, and we manually wrote test code to attempt to discern the impact of, e.g., changing a condition from `if (!handler)` to `if (handler === undefined)`. Such a mutant is equivalent only if handler can never be any other “falsy” value (e.g., `null`, `false`, `NaN`, `0`, or the empty string”).

We labeled a mutant as *not equivalent* if it produced a change that could be observed as a behavioral change to a user of the library. By necessity, this definition is conservative: if there could exist any client of the library that would witness a different behavior, then the mutant is not equivalent. Note that this definition also includes changes to output messages printed on the console or to the messages included with errors. For example, a mutant in the statement `const hours = Math.floor(totalMinutes/60)` that changes the call from `Math.floor` to `Math.round` will result in the value of hours being incorrect. Of course, if hours is never used (or it doesn’t matter that it is off-by-one), then the mutant could still be equivalent. Hence, we also found it necessary to trace through code to determine that some mutants were not equivalent.

Coding began with a pilot phase, where each coder labeled 10 surviving mutants in each project as equivalent or not. This process demonstrated strong agreement (Cohen’s $\kappa = 0.873$) [30], and the coders met briefly to clarify the few disagreements before proceeding with the remainder of the dataset. After independently coding the remainder of the dataset, the two result files were again compared for inter-rater reliability agreement, finding $\kappa = 0.846$, again indicating strong agreement [30]. To finalize the coding, the two authors met to discuss the cases on which they disagreed, reaching a consensus on the coding for all mutants during a single 30-minute session.

The results are shown in Table IV. Of the 524 *LLMorpheus* mutants examined, the majority (418, or 80%) are “not equivalent” and 106 (20%) are “equivalent”. To contextualize these results, we also examine the mutants generated by *StrykerJS*

TABLE IV
NUMBER OF EQUIVALENT SURVIVING MUTANTS GENERATED BY
LLMORPHEUS AND *STRYKERJS*

Project	<i>LLMorpheus</i>		<i>Stryker.js</i>	
	Equiv	Not Equiv	Equiv	Not Equiv
<i>Complex.js</i>	6	44	1	49
<i>countries-and-timezones</i>	18	11	0	6
<i>crawler-url-parser</i>	11	39	5	45
<i>delta</i>	9	41	7	43
<i>image-downloader</i>	4	13	0	8
<i>node-dirty</i>	13	37	0	50
<i>node-geo-point</i>	3	47	5	45
<i>node-jsonfile</i>	12	25	0	3
<i>plural</i>	12	38	0	37
<i>pull-stream</i>	1	49	0	50
<i>q</i>	0	50	0	50
<i>spacel-core</i>	17	24	2	18
<i>zip-a-folder</i>	0	0	0	6
<i>Total</i>	106	418	20	410

and found that of 430 surviving mutants, 410 (95%) are “not equivalent” and 20 (5%) are “equivalent”.

We further examined the 106 equivalent *LLMorpheus* mutants and observed several common patterns, including: (i) checking for `null`-ness or `undefined`-ness in different ways (e.g., replacing `x != null` with `!x` or vice versa), (ii) refactoring of calls to the `String.substring` method with one of its near-equivalent counterparts `String.substr` and `String.slice`, (iii) adding modifiers such as `/g` or `/m` to a regular expression in cases where this does not have any effect, (iv) calls to the `Array.slice` method in cases where this does not have any effect, and (v) calling functions with more arguments than are declared. For the 106 equivalent mutants under consideration, approximately 40% fall into one of these categories. We expect that most of these equivalent mutants can be filtered out using an AST-based static analysis. However, further investigation is needed because some mutants that cause behavioral differences are syntactically similar to these patterns. This means that any pattern-matching-based approach should consider the context in which the mutation occurs to determine whether a mutant is likely equivalent. Section VII will discuss future work to reduce the number of equivalent mutants.

The majority (80%) of the surviving mutants produced by *LLMorpheus* are not equivalent to the original code fragments they replace. *LLMorpheus* produces significantly more “equivalent” mutants than *StrykerJS*. However, the number of “not-equivalent” mutants exceeds the number of equivalent mutants by more than a factor of three, and preliminary analysis reveals good potential for future work on automatically filtering out equivalent mutants using static analysis.

¹³For projects with fewer than 50 surviving mutants, we used as many as were available.

E. RQ3: What Is the Effect of Different Temperature Settings?

To explore the impact of an LLM's temperature setting, we repeated the experiment with the *codellama-34b-instruct* LLM using temperatures 0.25, 0.50, and 1.0. The results of these experiments are summarized in Table III. As can be seen from the table, the total number of mutants and the number of surviving mutants at temperatures 0.0, 0.25, and 0.50 are generally somewhat similar. However, at temperature 1.0, both the total number of mutants and the number of surviving mutants decline noticeably compared to the results for temperature 0.0. Inspection of the results revealed that this is partly because more of the generated mutants are syntactically invalid.

A secondary question is how temperature affects the variability of results. To answer this question, we repeated the experiment 5 times at each temperature and measured how many distinct mutants occur and how many mutants occur in all five runs. We found that, at higher temperatures, the number of distinct mutants increases rapidly and that the number of mutants common to all runs decreases accordingly. For example, for *Complex.js*, *LLMorpheus* generates 1,217 distinct mutants at temperature 0.0 of which 1,181 (97.04%) are common to all five runs. At temperature 0.25, the number of distinct mutants increases to 2,354, of which 447 (18.99%) are common to all five runs. At temperature 0.5, there are 3,196 distinct mutants of which 205 (6.41%) are common to all runs. At temperature 1.0, there are 4,200 distinct mutants, of which 17 (0.4%) are common to all runs, meaning that, effectively, at temperature 1.0, each run produces completely different mutants. The results for the other subject applications are similar. The supplemental materials associated with this paper include an analysis showing the overall variability in mutants killed and survived across each of the five runs.

LLMorpheus generally produces similar numbers of mutants at temperatures ≤ 0.5 , of which a similar number survives. At temperature 1.0, the number of generated and surviving mutants declines noticeably because more candidate mutants are syntactically invalid. The variability of results is inversely dependent on the temperature, with mostly the same mutants being produced at temperature 0.0 and mostly different mutants at temperature 1.0 in different runs.

F. RQ4: What Is the Effect of Variations in the Prompting Strategy Used by *LLMorpheus*?

Thus far, we have evaluated the effectiveness of the prompt template of Fig. 7(a) (henceforth referred to as *full*) by measuring how many mutants are generated and classifying them as “killed”, “survived”, or “timed-out” (see Table II). To determine what the effect is of each component of this prompt, we experimented with the following variations¹⁴:

- a) *Onemutation*: This variant requests just one replacement of the placeholder instead of three possible replacements.
- b) *Noexplanation*: This variant omits the phrase “This would result in different behavior because <brief explanation>.”.

- c) *Noinstructions*: This variant omits the phrase “Please consider changes such as using different operators, changing constants, referring to different variables, object properties, functions, or methods.”

- d) *Genericsystemprompt*: In this variant, we replace the system prompt of Fig. 7(b) with a generic message “You are a programming assistant. You are expected to be concise and precise and avoid any unnecessary examples, tests, and verbosity.”

- e) *Basic*: This minimal template only asks the LLM to provide a code fragment with which the placeholder can be replaced without any additional context.

Table V shows, for each template, the total number of mutants and the number that were killed, survived, and timed out, respectively. From these results, it can be seen that:

- *full* and *genericsystemprompt* produced the most mutants and performed similarly, demonstrating that the use of a specialized system prompt has minimal impact,
- *noexplanation* and *noinstructions* produce only slightly fewer mutants and surviving mutants than *full* and *genericsystemprompt*, so including instructions or requesting explanations for suggested mutations has limited impact,
- using *onemutation* dramatically reduces the number of mutants from 6,712 to 2,333, demonstrating that it is helpful to request multiple suggestions, and
- using *basic* reduces the number of mutants to 1,326, suggesting that additional context in prompts is helpful.

We separately analyzed the variability of these results (Table V presents the results from a single trial) and found the number of mutants killed and survived to be quite stable across trials (the supplemental materials provide further detail).

We also investigated how similar mutants produced using the different prompt templates are to the original code fragments they replace. As manually inspecting sufficient samples of mutants from each configuration would be infeasible, we instead rely on an automated measure. We calculate the Levenshtein string edit distance for each mutant between the mutated and original code. Table VI reports the average string edit distance scores for each of the prompt templates by project.

Interpreting the results across different projects is challenging, as each project uses different code idioms that might lead to different mutations. However, we observe several interesting trends by comparing the mutant similarity across prompts (within the same project). We find the *basic* template to produce the mutants that are *least similar* to the original code. We examined samples of these mutants and found that many were creative changes that injected large code blocks in place of short, simple values. For example, in *crawler-url-parser*, the mutant with the most significant string edit distance (297) involves replacing a constant TRUE with an object literal. While the *onemutation* template tended to produce mutants most similar to the original code, this is likely due to the more limited sample space. We infer that prompting for multiple mutants can result in the LLM suggesting more significant code changes than it would otherwise have.

¹⁴All prompt templates are included with the supplemental materials.

TABLE V
NUMBER OF MUTANTS GENERATED USING THE *CODELLAMA-34B-INSTRUCT* LLM AT TEMPERATURE 0.0 USING TEMPLATES FULL, ONEMUTATION, NOEXPLANATION, NOINSTRUCTIONS, GEN.SYSTEM PROMPT, BASIC (SHOWING ONE RUN OF EACH)

	Full (run #312)				Onemutation (run #365)				Noexplanation (run #372)				Noinstructions (run #378)				GenericsystemPrompt (run #384)				Basic (run #390)			
	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout	#mutants	#killed	#survived	#timeout
<i>Complex.js</i>	1,199	725	473	1	406	245	161	0	1,125	676	448	1	1,137	696	440	1	1,199	740	458	1	185	120	65	0
<i>countries-and-timezones</i>	217	188	29	0	79	65	14	0	211	183	28	0	218	174	44	0	217	191	26	0	48	44	4	0
<i>crawler-url-parser</i>	285	157	128	0	86	50	36	0	239	140	99	0	246	134	112	0	246	143	103	0	67	49	18	0
<i>delta</i>	767	634	101	32	266	221	37	8	734	598	110	26	759	612	115	32	790	659	99	32	201	167	28	6
<i>image-downloader</i>	89	72	17	0	34	26	8	0	77	62	15	0	84	69	15	0	88	72	16	0	10	7	3	0
<i>node-dirty</i>	275	163	100	12	99	55	41	3	258	146	99	13	260	146	103	11	277	162	104	11	44	24	18	2
<i>node-geo-point</i>	302	223	79	0	104	74	30	0	297	216	81	0	306	230	76	0	305	229	76	0	62	54	8	0
<i>node-jsonfile</i>	154	49	48	57	57	18	18	21	152	54	45	53	148	45	51	52	150	49	49	52	22	11	3	8
<i>plural</i>	281	205	75	1	100	70	30	0	273	198	74	1	261	189	71	1	272	209	62	1	92	78	14	0
<i>pull-stream</i>	769	441	271	57	280	165	95	20	774	440	278	56	781	467	248	66	763	442	266	55	149	88	54	7
<i>q</i>	2,035	158	1,792	85	703	46	630	27	1,856	138	1,635	83	1,958	138	1,726	94	2,007	145	1,770	92	401	38	350	13
<i>spacel-core</i>	239	199	39	1	80	63	17	0	211	175	35	1	187	155	31	1	214	181	32	1	25	23	2	0
<i>zip-a-folder</i>	100	23	3	74	39	19	17	3	98	27	3	68	97	26	4	67	101	27	3	71	20	5	1	14
<i>Total</i>	6,712	3,237	3,155	320	2,333	1,117	1,134	82	6,305	3,053	2,950	302	6,442	3,081	3,036	325	6,629	3,249	3,064	316	1,326	708	568	50

TABLE VI
AVERAGE STRING SIMILARITY OF MUTANTS TO THE ORIGINAL CODE FRAGMENTS THAT THEY REPLACE, FOR MUTANTS GENERATED USING EACH OF THE PROMPT TEMPLATES AT TEMPERATURE 0.0 USING *CODELLAMA-34B-INSTRUCT*

	Full	Onemutation	Noexplanation	Noinstructions	Generic sys. prompt	Basic
<i>Complex.js</i>	4.27	3.37	5.09	4.27	4.17	11.98
<i>countries-and-timezones</i>	11.13	7.75	11.17	10.87	10.85	11.29
<i>crawler-url-parser</i>	9.50	6.41	9.46	9.49	9.30	20.04
<i>delta</i>	9.55	7.38	9.91	9.43	9.14	19.63
<i>image-downloader</i>	12.67	8.82	12.89	11.01	11.48	21.92
<i>node-dirty</i>	7.53	6.90	7.58	7.41	7.51	17.52
<i>node-geo-point</i>	8.86	6.10	8.79	7.75	8.66	15.66
<i>node-jsonfile</i>	9.73	6.98	9.76	7.77	8.91	11.64
<i>plural</i>	8.14	5.21	8.41	7.58	7.80	23.64
<i>pull-stream</i>	6.72	4.57	7.53	7.48	7.30	11.92
<i>q</i>	8.61	7.61	9.21	8.60	8.58	16.18
<i>spacel-core</i>	9.30	5.86	10.44	9.43	9.44	14.27
<i>zip-a-folder</i>	9.85	5.33	9.02	10.05	10.10	24.60

The *full* template produces the most mutants and surviving mutants overall. Using a specialized system prompt has a marginal effect. Including instructions on performing mutations and requesting explanations for mutations only modestly affects the number of mutants generated and their detection rate. Requesting only one mutation dramatically reduces the number of generated and surviving mutants, and even greater reductions are observed if the LLM is only asked to fill in the placeholder without additional guidance.

G. RQ5: How Does the Effectiveness of LLMorpheus Depend on the LLM Being Used?

The results discussed thus far were obtained with the *codellama-34b-instruct* LLM. To determine how the quality of results depends on the particular LLM being used; we also experimented with *codellama-13b-instruct*, *llama-3.3-70b-instruct*, *mixtral-8x7b-instruct*, and *gpt-4o-mini* at temperature 0.0.

Table VII shows the number of mutant candidates produced using each model (along with a breakdown how many of those

candidates are syntactically invalid, identical to the original code, or duplicates), and the number of mutants produced using each model, classified as killed, surviving, and timed-out. Fig. 9 shows a visual comparison of the total number of mutant candidates and mutants produced using each of the five LLMs under consideration, aggregated over all 13 subject applications. From these results, it can be seen that:

- The *codellama-34b-instruct* model generates the largest number of mutant candidates (9,967), though the number of mutant candidates produced by *codellama-13b-instruct*, *mixtral-8x7b-instruct*, *llama-3.3-70b-instruct*, and *gpt-4o-mini* are quite similar. *codellama-13b-instruct* produces noticeably fewer mutant candidates (8,088).
- All models produce a significant number of mutant candidates that is syntactically invalid, ranging from 3,703 in the case of *gpt-4o-mini* to 2,540 in the case of *mixtral-8x7b-instruct*.
- *codellama-13b-instruct* is the only model that produces a significant number of mutant candidates that are identical to the original code fragments that they replace (922).
- None of the models produces a significant number of duplicate mutant candidates.
- The number of mutants that remain after discarding the invalid, identical, and duplicate mutant candidates ranges from 5,402 in the case of *mixtral-8x7b-instruct* to 6,823 in the case of *llama-3.3-70b-instruct*, with *codellama-34b-instruct* producing almost as many valid mutants (6,712).
- *llama-3.3-70b-instruct* produces the most surviving mutants (3,423), followed by *codellama-34b-instruct* (3,155).

We also explored the variability of results produced using *codellama-13b-instruct*, *mixtral-8x7b-instruct*, *llama-3.3-70b-instruct* and *gpt-4o-mini* by conducting each experiment 5 times, and determined how many distinct mutants are produced and how many mutants occur in all five runs. We found that, at temperature 0.0, the results obtained with *codellama-13b-instruct* are very stable across runs, with 96.15%–100% of all mutants occurring in each of the five runs. However, with *mixtral-8x7b-instruct*, *llama-3.3-70b-instruct*, and *gpt-4o-mini*, we encountered more variability. With *mixtral-8x7b-instruct*,

TABLE VII
MUTANTS GENERATED WITH THE *CODELLAMA-13B-INSTRUCT*, *MIXTRAL-8X7B-INSTRUCT*, *LLAMA-3.3-70B-INSTRUCT*, AND *GPT-4O-MINI* LLMs, USING THE FOLLOWING PARAMETERS: TEMPERATURE: 0.0, MAXTOKENS: 250, TEMPLATE: *TEMPLATE-FULL.HB*, SYSTEMPROMPT: *SYSTEMPROMPT-MUTATIONTESTINGEXPERT.TXT*

	<i>codellama-13b-instruct</i> (run #354)								<i>mixtral-8x7b-instruct</i> (run #360)							
	#candidates	#invalid	#identical	#duplicate	#mutants	#killed	#survived	#timeout	#candidates	#invalid	#identical	#duplicate	#mutants	#killed	#survived	#timeout
<i>Complex.js</i>	1,410	339	116	28	955	553	401	1	1,272	310	0	15	962	589	373	0
<i>countries-and-timezones</i>	305	83	15	1	207	177	30	0	272	65	2	5	205	166	39	0
<i>crawler-url-parser</i>	494	186	51	12	247	129	118	0	411	165	0	3	234	130	104	0
<i>delta</i>	1,334	530	92	16	712	583	107	22	1,132	452	0	24	680	516	128	36
<i>image-downloader</i>	122	40	5	2	77	48	29	0	107	38	0	1	69	46	23	0
<i>node-dirty</i>	439	161	33	11	245	142	92	11	300	109	0	10	191	111	72	8
<i>node-geo-point</i>	390	64	21	16	304	237	67	0	341	88	0	11	247	166	81	0
<i>node-jsonfile</i>	191	43	10	7	138	43	45	50	155	23	0	4	132	54	32	46
<i>plural</i>	407	100	99	17	208	154	53	1	299	73	0	8	226	166	60	0
<i>pull-stream</i>	1,002	279	54	13	669	386	237	46	934	255	1	6	678	386	248	44
<i>q</i>	2,993	901	379	55	1,713	122	1,518	73	2,418	772	3	50	1,643	112	1,460	71
<i>spacl-core</i>	377	142	40	7	185	160	25	0	330	152	0	3	157	134	22	1
<i>zip-a-folder</i>	137	43	7	1	87	27	55	5	117	38	0	0	78	24	44	10
<i>Total</i>	9,601	2,911	922	186	5,582	2,761	2,777	209	8,088	2,540	6	140	5,402	2,600	2,686	216
	<i>llama-3.3-70b-instruct</i> (run #23)								<i>gpt-4o-mini</i> (run #58)							
	#candidates	#invalid	#identical	#duplicate	#mutants	#killed	#survived	#timeout	#candidates	#invalid	#identical	#duplicate	#mutants	#killed	#survived	#timeout
<i>Complex.js</i>	1,417	279	0	53	1,138	690	447	1	1,432	446	0	38	986	596	390	0
<i>countries-and-timezones</i>	304	64	0	14	240	207	33	0	308	99	0	9	209	171	38	0
<i>crawler-url-parser</i>	506	175	0	22	319	208	111	0	516	227	0	11	275	181	94	0
<i>delta</i>	1,333	539	0	54	794	626	130	38	1,345	636	2	40	707	564	108	35
<i>image-downloader</i>	122	43	0	5	79	54	25	0	123	58	0	3	65	45	20	0
<i>node-dirty</i>	453	121	1	10	331	168	142	21	453	188	0	9	265	154	103	8
<i>node-geo-point</i>	399	39	0	22	358	255	103	0	399	86	0	20	311	225	86	0
<i>node-jsonfile</i>	198	25	0	6	173	64	37	72	198	44	0	6	154	64	26	64
<i>plural</i>	427	96	0	29	331	244	87	0	428	110	5	26	313	257	55	1
<i>pull-stream</i>	1,044	262	0	10	782	465	265	52	1,037	302	0	16	735	420	247	68
<i>q</i>	3,074	855	0	80	2,219	127	2,006	86	3,084	1,287	2	69	1,795	137	1,597	61
<i>spacl-core</i>	383	134	0	18	236	203	32	1	392	158	0	10	215	195	20	0
<i>zip-a-folder</i>	145	24	0	2	121	87	5	29	143	62	0	4	81	14	7	60
<i>Total</i>	9,805	2,656	1	325	6,823	3,398	3,423	300	9,858	3,703	9	261	5,885	3,023	2,791	297

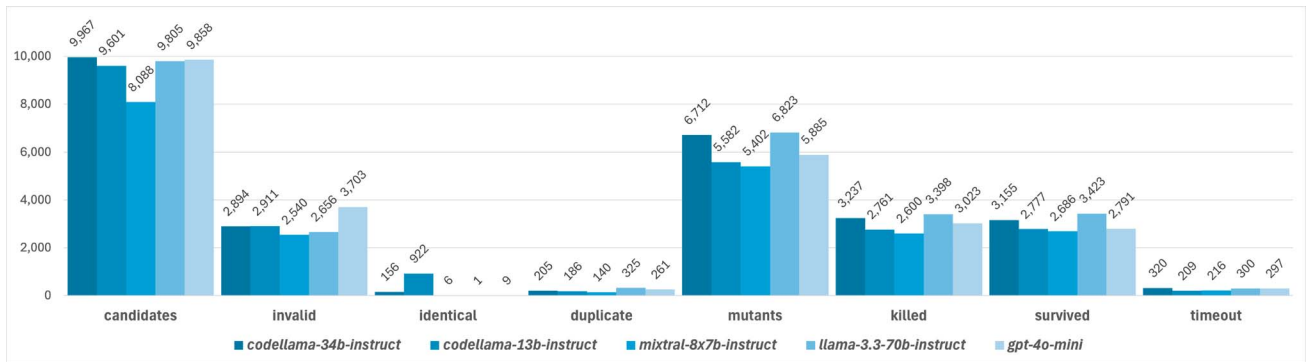


Fig. 9. Comparison of the number of mutant candidates and mutants generated with the *codellama-13b-instruct*, *mixtral-8x7b-instruct*, *llama-3.3-70b-instruct*, and *gpt-4o-mini* LLMs at temperature 0.0. This chart was created from the data shown in Tables III and VII.

between 34.22%–50% of mutants occur in all five runs, with *llama-3.3-70b-instruct*, between 28.26%–58.94% occur in all five runs, and with *gpt-4o-mini*, between 28.26%–58.94%. Fig. 10 visualizes the variability of *LLMorpheus* across all configurations. The figure shows, in the aggregate, for all 13 subject applications, in how many runs each mutant was observed. We also analyzed the variance of the number of mutants killed and

survived, finding that the mutation score was relatively stable despite the diversity of mutants across trials. The supplemental materials include tables showing the average and standard deviation of the number of mutants killed and survived.

We also examined the string similarity of mutants produced by the five LLMs to the original code and found that the *mixtral-8x7b-instruct* model tends to generate mutants with the greatest

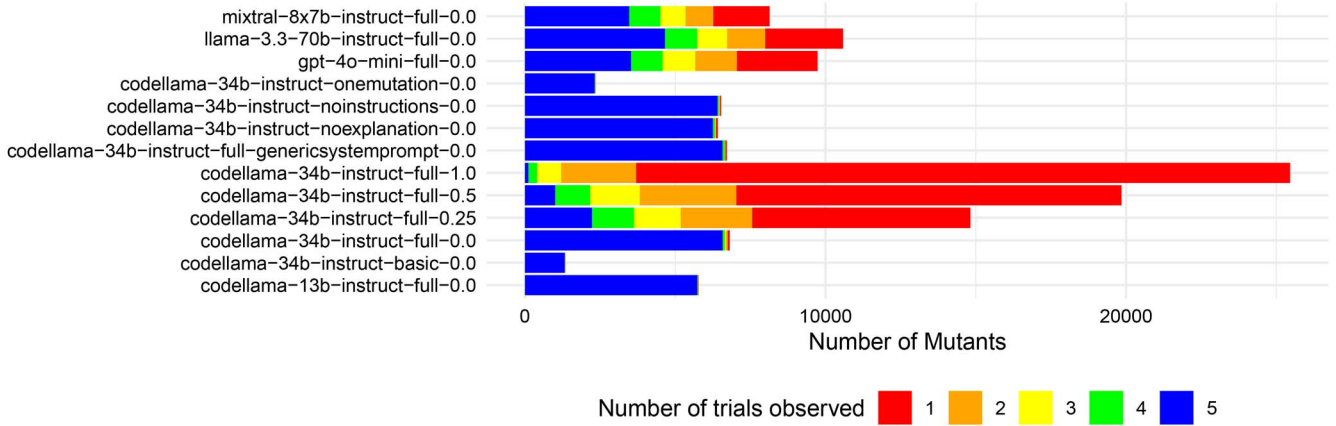


Fig. 10. Variability of mutants generated by *LLMorpheus*. For each replacement generated at each position, we count the number of trials (of 5 total) where that replacement was generated.

string edit distance in the most projects. We examined the top 2 mutants with the greatest string edit distance generated by this model for each project, finding several cases of unusual completions. In *q*, mixtral’s most dissimilar mutants (distance 219) replaced a string literal that referred to the function “allResolved” with a declaration of the same function. In *delta*, mixtral’s most dissimilar mutants (distance 155) apply a reduce operation to an object before invoking Object.keys on it. We saw similar trends for mixtral across all projects, with mutants that tended to include long code declarations. Examining the mutants with the greatest string edit distance for the other four LLMs, we did not find significant trends that held across all targets. Further details can be found in the supplemental materials.

All five LLMs under consideration can successfully generate large numbers of (surviving) mutants. *llama-3.3-70b-instruct* and *codellama-34b-instruct* tend to produce the largest number of surviving mutants, and *codellama-34b-instruct* produces stable results across experiments when temperature 0.0 is used. *llama-3.3-70b-instruct*, *mixtral-8x7b-instruct*, and *gpt-4o-mini* produce highly variable results, even at temperature 0.0.

H. RQ6: What Is the Cost of Running LLMorpheus?

The primary costs of running *LLMorpheus* are the time required to run experiments and the expenses associated with LLM usage. Regarding the latter, for the experiments reported on in this paper, we have relied on several commercial LLM service providers (octo.ai, openrouter.ai, and openai.com). Such costs tend to vary depending on the provider and the LLM being used and are typically calculated as a function of the number of “tokens” used in the prompt and the completion¹⁵. The cost of commercial LLM providers also tends to vary over time, and when a newer version of an LLM is released, it often costs the same as the older version that it replaced. In our experiments, the total number of tokens used for running a full experiment with *LLMorpheus* varied by less than 20% for the five LLMs

¹⁵Depending on the provider, the number of requests may also incur additional costs, though that was not the case for our experiments.

TABLE VIII

RESULTS FROM LLMORPHEUS EXPERIMENT (RUN #312). MODEL: *CODELLAMA-34B-INSTRUCT*, TEMPERATURE: 0.0, MAXTOKENS: 250, TEMPLATE: *TEMPLATE-FULL.HB*, SYSTEMPROMPT: *SYSTEMPROMPT-MUTATIONTESTINGEXPERT.TXT*

Project	Time (sec)		#Tokens		
	<i>LLMorpheus</i>	<i>StrykerJS</i>	prompt	compl.	total
<i>Complex.js</i>	3,050.00	637.85	967,508	102,517	1,070,025
<i>countries-and-timezones</i>	1,070.89	313.86	105,828	23,441	129,269
<i>crawler-url-parser</i>	1,642.70	929.43	386,223	39,175	425,398
<i>Delta</i>	2,961.66	3,839.60	890,252	98,974	989,226
<i>image-downloader</i>	430.53	379.25	24,655	9,134	33,789
<i>node-dirty</i>	1,526.20	241.81	246,248	33,070	279,318
<i>node-geo-point</i>	1,411.11	987.17	316,333	30,013	346,346
<i>node-jsonfile</i>	690.61	474.78	57,516	14,797	72,313
<i>Plural</i>	1,521.32	155.24	265,602	34,174	299,776
<i>pull-stream</i>	2,492.50	1,608.97	208,130	76,513	284,643
<i>Q</i>	5,241.46	14,034.67	2,127,655	220,215	2,347,870
<i>spacel-core</i>	1,351.08	798.96	162,705	29,236	191,941
<i>zip-a-folder</i>	500.57	1,156.11	82,457	10,725	93,182
Total	23,890.64	25,557.70	5,841,112	721,984	6,563,096

that we used¹⁶, suggesting that token usage is a reasonable proxy for the financial costs incurred. For these reasons, we use the number of input and output tokens used in our experiments as the primary cost metric for evaluating *LLMorpheus*’s LLM usage. For completeness, we also discuss the expense in US dollars at the time of running the experiments below, but the reader should be aware that these costs are likely to vary over time.

The **time** column in Table VIII shows the time needed to run *LLMorpheus* and the modified version of *StrykerJS* on each subject application. As can be seen in the table, *LLMorpheus* requires between 430.53 seconds (about 7 minutes) and 5,241.46 seconds (about 87 minutes) and *StrykerJS* between 155.24 seconds (about 2.5 minutes) and 14,034.67 seconds (about 234 minutes).

The last three columns of Table VIII show the number of tokens used in prompts and completions for each subject application and in the aggregate. From these results, it can be seen that running *LLMorpheus* required between 24,655

¹⁶Calculated from the total number of tokens reported in the Supplemental Materials associated with this paper for experiments using the “full” prompt template at temperature 0.0.

and 2,127,655 prompt tokens and between 9,134 and 220,215 completion tokens. Hence, in the aggregate, 5,841,112 prompt tokens and 721,984 completion tokens were required. At the time of conducting the experiments, the cost of the *codellama-34b-instruct* LLM using octo.ai's LLM service was \$0.50 per million input tokens and \$1.00 per million output tokens, so for running *LLMorpheus* on all 13 applications, a total cost of approximately \$3.62 was incurred. Moreover, at the time of conducting our experiments, the *llama-3.3-70b-instruct* model that we used can be accessed from \$0.12 per million input tokens and \$0.30 per million output tokens at openrouter.ai, and the *gpt-4o-mini* model that we used can be accessed from \$0.15 per million input tokens and \$0.60 per million output tokens from openai.com. Hence, a full experiment can be run for less than \$1 with *llama-3.3-70b-instruct*, and for approximately \$1.30 with *gpt-4o-mini*.

It should be pointed out that the cost of the LLMs we used is significantly lower than that of larger state-of-the-art proprietary LLMs such as OpenAI's *gpt-4o*, for which <https://openai.com/pricing> quotes a cost of \$2.50 per million input tokens and \$10 per million output tokens at the time of writing. While such models might be even more capable of suggesting useful mutants, it is encouraging to see that lower-cost LLMs can achieve good results.

LLMorpheus requires between 7 and 87 minutes to generate mutants for 13 subject applications. At the time of conducting our experiments, a full experiment with *LLMorpheus* on all 13 applications costs up to \$3.62 depending on the LLM being used, suggesting that cost is not a prohibitive limiting factor.

I. RQ7: Is *LLMorpheus* Capable of Producing Mutants That Resemble Existing Bugs?

To determine whether *LLMorpheus* is capable of producing mutants that resemble existing bugs, we conducted a case study involving 40 real-world bugs, shown in Table IX. The construction of this dataset was previously discussed in Section IV-B(d). In this study, we applied *LLMorpheus* to the *fixed* version of a program by introducing placeholders near the location of the fix, generating mutants, executing the program's tests for each of these mutants, and checking if the observed test failures were identical to those caused by the original bug. We considered two failures to be the same if the same error message and stack trace were produced. This task involves significant manual effort and time as it involves executing all tests for each mutant and manually comparing the behavior of the test failures caused by mutants against test failures caused by the original bug. Given the potential non-determinism inherent to *LLMorpheus*, we repeat this mutant generation, test execution and manual inspection process a total of five times per-bug. Our artifact contains complete details on each of the bugs examined, showing the buggy code and mutant side-by-side, along with our comments and analysis.

For each bug, Table IX shows the name and description of the package, the issue number associated with the bug in the repository's issue tracker, and the commit ID (SHA) containing

the fix. Further details about these bugs, the mutants produced by *LLMorpheus*, and the test results obtained with each mutant are included in our artifact.

The last three columns of the table show, for each bug, to what extent the mutants produced by *LLMorpheus* mimic the original bug. We classify the bugs into the following three categories:

same code change(s). This means that *LLMorpheus* generates at least one mutant that contains the same code change(s) as the bug *and* causes the same test failures,

same test failure(s). This means that *LLMorpheus* generates at least one mutant that is not syntactically the same as the bug but results in the same test failure(s) as the bug (possibly also causing other tests to fail), and

different test failures. This means that *LLMorpheus* does not generate at least one mutant that causes the same test failure as the bug.

As can be seen from the table, for 10 of the 40 bugs, *LLMorpheus* produced mutants consisting of code fragments that are syntactically identical to the original bug and that caused the same test failures. Moreover, in an additional 26 cases, *LLMorpheus* produced mutants that caused the same test failures as the ones caused by the original bug. In only 4 cases did *LLMorpheus* not produce any mutants that cause similar test failures as the original bugs. In addition, it should be noted that, of the 40 bugs under consideration, 35 involved a patch that involved complex changes that do not correspond to the application of a traditional mutation operators (e.g., changing conditions by adding/removing subconditions, referencing different variables, calling different/additional functions, adding arguments in function calls, changing regular expression literals, etc.) This means that a traditional mutation testing tool such as *StrykerJS* would be unable to reproduce these bugs exactly (though it might still be able to create mutants that produce the same failures).

Our artifact contains details regarding all of the bugs that we studied. Below, we report on our findings for four of the bugs in more detail.

a) *Express.js Bug#2*: Fig. 11 shows the patch for bug #2 in Express, a popular web framework for Node.js. This bug occurs at line 361 in the file `lib/request.js` and involves the invocation of a function `trust` with a single argument `this.connection.remoteAddress`. Here, the fix involved the addition of a second argument, `0`. Reintroducing this bug in the fixed version causes two tests to fail. When applied to the fixed version, *LLMorpheus* creates the following three mutants:

- replacing `!trust(this.connection.remoteAddress, 0)` with `trust(this.connection.remoteAddress, 1)`
- replacing `!trust(this.connection.remoteAddress, 0)` with `!trust(this.connection.remoteAddress)`
- replacing `!trust(this.connection.remoteAddress, 0)` with `trust(this.connection.localAddress, 0)`

The second mutant is identical to the original bug. The other two mutants cause multiple test failures that differ from those caused by the original bug.

TABLE IX

RESULTS OF CASE STUDY INVESTIGATING WHETHER *LLMORPHEUS* CAN GENERATE MUTANTS THAT ARE SIMILAR TO REAL BUGS. EACH ROW OF THE TABLE CORRESPONDS TO ONE BUG, FOR WHICH THE FIRST FOUR COLUMNS OF THE TABLE STATE THE NAME AND DESCRIPTION OF THE PACKAGE, ISSUE/BUG NUMBER, AND COMMIT ID (SHA) CONTAINING THE BUG FIX. THE LAST THREE COLUMNS CLASSIFY EACH BUG INTO ONE OF THE FOLLOWING CATEGORIES: “SAME CODE CHANGE(S)” MEANS THAT *LLMORPHEUS* GENERATES AT LEAST ONE MUTANT THAT CONTAINS THE SAME CODE CHANGE(S) AS THE BUG AND CAUSES THE SAME TEST FAILURE(S), “SAME TEST FAILURE(S)” MEANS THAT *LLMORPHEUS* GENERATES AT LEAST ONE MUTANT THAT IS NOT SYNTACTICALLY THE SAME AS THE BUG BUT CAUSES THE SAME TEST FAILURE(S) (POSSIBLY ALSO CAUSING OTHER TESTS TO FAIL), AND “DIFFERENT TEST FAILURES” MEANS THAT *LLMORPHEUS* DOES NOT GENERATE AT LEAST ONE MUTANT THAT CAUSES THE SAME TEST FAILURE(S)

package	description	issue/bug #	SHA	same code change(s)	same test failure(s)	different test failure(s)
css-loader	CSS files	663	d1d8221		✓	
css-loader	CSS files	789	e3bb83a		✓	
css-loader	CSS files	1036	ded2a79		✓	
css-loader	CSS files	1261	729a314			✓
compression	file compression	170	b7d5d77			✓
countries-and-timezones	accessing country/timezone data	60	97a106f	✓		
express.js	web application framework	2	—	✓		
fast-glob	file system	223	05a4c08		✓	
fast-glob	file system	391	eb55d1d		✓	
fast-xml-parser	XML parsing	234	ea5d544		✓	
fast-xml-parser	XML parsing	595	b0ea635		✓	
fs-extra	file system	190	e05c685		✓	
fs-extra	file system	291	2e7f755		✓	
fs-extra	file system	679	7c251d6		✓	
hessian	serialization	4	—		✓	
hexo	blogging framework	12	—	✓		
htmlparser2	HTML parsing	746	214ab08		✓	
htmlparser2	HTML parsing	913	04c411c	✓		
jsdiff	file comparison	94	d76ac52		✓	
jsdiff	file comparison	118	4a899c0		✓	
jsdiff	file comparison	217	6464b29			✓
jsdiff	file comparison	493	f38e47d		✓	
karma	testing framework	4	—	✓		
memfs	in-memory file system	59	b90c016		✓	
memfs	in-memory file system	391	301f2d1		✓	
memfs	in-memory file system	853	8b021b3		✓	
memfs	in-memory file system	870	7c5999c	✓		
memfs	in-memory file system	1024	711c4bd	✓		
memfs	in-memory file system	1093	ede0f4f	✓		
node-jsonfile	reading/writing JSON files	24	c2c8a2c	✓		
node-jsonfile	reading/writing JSON files	25	afaba5d		✓	
normalize-url	URL utilities	38	6078d91		✓	
normalize-url	URL utilities	82	191ad4b		✓	
simple-statistics	statistics	334	522a716		✓	
simple-statistics	statistics	633	6547df7		✓	
yargs	command-line arguments	1364	35d777c		✓	
yargs	command-line arguments	1376	3d26d11	✓		
yargs	command-line arguments	1422	9a42b63			✓
yargs	command-line arguments	1493	63b3dd3		✓	
yargs	command-line arguments	2171	f91d9b3		✓	
Total: 40				10	26	4

```

360 360
361 - if (!trust(this.connection.remoteAddress)) {
361 + if (!trust(this.connection.remoteAddress, 0)) {
362 362     return proto;
363 363 }

```

Fig. 11. Patch corresponding to bug #2 in *express.js* [27].

```

301 301 proto.writeObject = function (obj) {
302 - if (is.nullOrUndefined(obj)) {
302 + if (is.nullOrUndefined(obj) ||
303 + // : { a: { '$class': 'xxx', '$': null } }
304 + (is.string(obj.$class) && is.nullOrUndefined(obj.$))) {
303 305     debug('writeObject with a null');

```

Fig. 12. Patch corresponding to bug #4 in *hessian.js* [27].

Given the possibility of non-determinism impacting this experiment, we conducted five repeated trials¹⁷. We found that in some cases, *LLMorpheus* produces mutants such as `!trust(this.connection.localAddress, 1)` that differ from the original bug but cause the same test failures. Moreover, in one experiment, *LLMorpheus* produced a mutant `!this.app.get('trust proxy')` that

¹⁷Data for five experiments with each of the 40 bugs is included with supplemental materials.

reproduces one of the two test failures caused by the original bug.

b) Hessian.js Bug#4: Fig. 12 shows the patch for bug #4 in *Hessian*, a serialization framework. This bug occurs at line 302 in file `lib/v1/encoder.js` and involves the condition of an `if`-statement. Here, the fix for the bug involves changing the condition from `is.nullOrUndefined(obj)` to `is.nullOrUndefined(obj) || (is.string(obj.$class) && is.nullOrUndefined(obj.$))`. Reintroducing this bug in the fixed version results in a test failure. When

```

17 17    dirent.mode = mode;
18 -    dirent.path = link.getPath();
18 +    dirent.path = link.getParentPath();
19 19

```

Fig. 13. Patch corresponding to issue #1024 in *memfs*.

```

104 104
105 -    if (commandKeys.length > 0) {
105 +    if ((currentContext.commands.length > 0) || (commandKeys.length > 0)) {
106 106        argv._.slice(currentContext.commands.length).forEach((key) => {

```

Fig. 14. Patch corresponding to issue #1364 in *yargs*.

applied to the fixed version, *LLMorpheus* creates the following 3 mutants:

- replacing `is.nullOrUndefined(obj) || (is.string(obj.$class) && is.nullOrUndefined(obj.$))` with `obj === null`,
- replacing `is.nullOrUndefined(obj) || (is.string(obj.$class) && is.nullOrUndefined(obj.$))` with `is.nullOrUndefined(obj.$)`
- replacing `is.nullOrUndefined(obj) || (is.string(obj.$class) && is.nullOrUndefined(obj.$))` with `!obj`

In this case, none of the generated mutants are identical to the original bug. However, the first and the third mutants *cause exactly the same test failures as the original bug*. The second mutant causes multiple test failures that differ from those produced by the original bug. We repeated the same experiment four more times, and while *LLMorpheus* never reproduced the original bug, it produced mutants with the same behavior as the original bug on multiple occasions.

c) *memfs* issue#1024: Fig. 13 shows the bug fix for issue #1024 in *memfs*, an in-memory file system for Node.js. Reintroducing the bug in the patched version results in failures in three tests for the `readdirsync` function. Here, the fix involves replacing a method call `link.getPath()` on line 18 in file `src/DirEnt.ts` with a call `link.getParentPath()`.

When applied to this line, *LLMorpheus* creates the following three mutants:

- replacing `link.getParentPath` with `link.getPath`,
- replacing `link.getParentPath` with `link.getName`, and
- replacing `link.getParentPath` with `""`.

The first mutant is identical to the original bug, the second mutant results in code that violates TypeScript's typing rules, and the third mutant causes three test failures that differ from those caused by the original bug. We repeated the experiment four more times and observed that the original bug was reproduced during one of the other runs.

d) *yargs* issue#1364: Fig. 14 shows the bug fix for issue #1364 in *yargs*, a popular framework for parsing command-line arguments. Reintroducing the bug in the patched version results in a single test failure. Here, the fix involves changing the condition of an `if` statement on line 105 in file `lib/validation.js` from `commandKeys.length > 0` to `(currentContext.commands.length > 0) || (commandKeys.length > 0)`.

Here, *LLMorpheus* creates three mutants:

- replacing `(currentContext.commands.length > 0) || (commandKeys.length > 0)` with `(currentContext.commands.length > 0) || (commandKeys.length > 0)`,
- replacing `(currentContext.commands.length > 0) || (commandKeys.length > 0)` with `(currentContext.commands.length === 0) && (commandKeys.length === 0)`, and
- replacing `(currentContext.commands.length > 0) || (commandKeys.length > 0)` with `argv._.length > 0`.

The first of these mutants causes two test failures, of which one is identical to the test failure caused by the original bug, the second mutant causes five test failures, of which one is identical to the test failure caused by the original bug, and the third mutant survives, i.e., it does not cause any test failures. We repeated the experiment four more times, and each time, at least one mutant was produced that triggered the same test failure as the original bug, along with a few additional test failures.

For the 40 bugs under consideration in the case study, *LLMorpheus* was able to produce mutants that are syntactically identical to the buggy code fragments in 10 cases, and mutants that produce the same test failures as the original bug in an additional 26 cases. This provides evidence that *LLMorpheus* is capable of generating mutants whose behavior resembles that of real-world bugs and that this capability is not entirely due to training-set leakage.

J. Experimental Data

All experimental data associated with the experiments reported on in this paper can be found at <https://github.com/neu-se/mutation-testing-data>.

V. THREATS TO VALIDITY

The projects used to evaluate *LLMorpheus* may not be representative of the entire ecosystem of JavaScript packages. To mitigate this risk, we select popular packages used in prior JavaScript testing tool evaluations and report results per project, discussing the full range of behaviors we witness. As in many evaluations of LLM-based tools, the validity of our conclusions may be threatened by including our evaluation subjects in the training data for the models. If the model were trained on bugs in some of the programs we asked it to create bugs in, one would expect its performance on those programs to vary significantly from those on which it was not pre-trained. We mitigate this risk by conducting experiments with five LLMs, four of which are "open" in the sense that the training process is documented, thus enabling reproducibility and detailed analysis of experimental results.

Truly determining if a mutant is equivalent requires significant effort and despite the best efforts of two authors to evaluate them rigorously, there may be errors in categorizing mutants. We interpret the high degree of inter-rater reliability ($\kappa = 0.846$) as a reasonable assurance of the reliability of this process.

One of the key evaluation criteria used in previous work on mutation testing is “coupling”, i.e., determining whether a test suite that detects particular mutants also detects particular real faults [5], [6], [26]. We investigated the feasibility of conducting such a study using the Bugs.js suite [27], but we found that most of these subjects could not be used at all due to their reliance on outdated versions of various libraries and because of their incompatibility with modern Node.js versions that *StrykerJS* requires, causing them to be incompatible with *LLMorpheus*. These projects also have flaky tests, making it particularly challenging to perform mutation analysis [28]. We, therefore, opted for conducting a case study involving 40 real-world bugs, including 4 real-world bugs from the Bugs.js suite that we were able to reproduce reliably and an additional 36 bugs taken from a variety of real-world Node.js applications for which we manually identified an issue in the project’s issue tracker that reported the problem and a subsequent bugfix commit. For these 40 bugs, *LLMorpheus* produced mutants that replicated the code changes from the original bug in 10 cases, and it produced mutants that replicated the test failures caused by the original bug in an additional 26 cases, suggesting that *LLMorpheus* can produce mutants that behave similarly to existing bugs in most cases. The results of this case study may be skewed because the code for previous buggy versions of the applications may have been included in the training set of the LLM that we used. However, the fact that *LLMorpheus* frequently produced mutants in the case study that *differed from the original bug but caused the same test failures* suggests that *LLMorpheus*’s ability to produce mutants that resemble real-world bugs is not entirely due to training-set leakage. The results of the case study may also have been skewed by the selection of the subject applications in the case study and by our focus, for pragmatic reasons, on bugs for which the fix involves a small number of lines of code.

As a deliberate design choice, *LLMorpheus* employs a fixed strategy for introducing placeholders, as illustrated in Fig. 6, which precludes the creation of mutants at certain locations, thus potentially limiting its effectiveness. However, traditional mutation testing tools such as *StrykerJS* are similarly limited by applying mutations only in selected locations and are *additionally limited* by restricting mutations to a fixed repertoire of mutation operators. Moreover, our current placeholder strategy has been shown to be effective at producing large numbers of (surviving) mutants and at producing mutants that resemble real-world bugs. Exploring mechanisms that allow users to specify placeholder schemes, e.g., as a predicate on AST nodes, is a topic for future work.

Evaluating tools that rely on LLMs face significant reproducibility challenges. We mitigate these risks by (i) evaluating *LLMorpheus* using four open LLMs that are version-controlled and permanently archived (in addition to one popular proprietary LLM), (ii) repeating each experiment 5 times and (iii) making all experimental data available as supplemental materials, and (iv) making *LLMorpheus*, our evaluation scripts and results publicly available. Including all results for all experiments in the main body of this paper would significantly decrease the readability of the work. Where we observed significant

variability in results, we include data regarding that distribution in the paper directly. In all cases, the supplemental materials associated with this paper include results for all trials of all experiments and summary tables that describe the observed variability for each configuration.

Lastly, a possible concern is that *LLMorpheus* only supports JavaScript and TypeScript, and its applicability beyond these languages may be unclear. Implementing the same approach for a different language would involve various steps (parsing ASTs, executing tests, etc.) that are language-specific and would involve significant engineering effort, but should otherwise be straightforward.

VI. RELATED WORK

Mutation testing, first introduced in the 1970’s [4], has a long history [31]. The era of “big code” and software repository mining has enabled the large-scale evaluation of the core hypothesis behind mutation testing: mutants are coupled to real faults. Just *et al.* mined real faults from Java applications and found a statistically significant correlation between mutation detection and real fault detection [5]. This finding has since been replicated on newer, larger datasets of faults from even more Java programs [6]. Gay and Salahirad extended this methodology to examine the extent to which individual mutation operators are most coupled to real faults [26]. While this has demonstrated that test suites that detect more mutants are also likely to detect more bugs, it also underscores the need for new mutation approaches that can generate faults coupled to more real bugs.

ML for Mutation Testing: Several recent projects have considered using LLMs and other AI-based techniques for mutation testing. μ Bert [32], [33] resembles *LLMorpheus* in that both techniques select some designated code fragments, and query a model what they could be replaced with. μ Bert masks one token at a time, so its mutations involve changes to a single variable or operator. By contrast, *LLMorpheus*’ placeholders correspond to (sequences of) AST nodes, so it may suggest mutations involving more significant changes to complex expressions. A crucial difference between the techniques is that *LLMorpheus* utilizes prompts that provide an LLM with additional guidance, whereas μ Bert provides no way of guiding the mutations at all and is, therefore, completely at the mercy of what the model thinks masked tokens should be replaced with. In our experiments with different prompts (Section IV-F), the *basic* prompt is analogous to μ Bert in that it merely asks the LLM what placeholders should be replaced with. Our results show this to be much less effective at producing interesting mutants, thus demonstrating the usefulness of including additional information in prompts. Our work also differs from [32] by considering several LLMs and different temperatures and targeting a different language.

In recent work, Garg et al. [34] explore the coupling between mutants generated using μ Bert and 45 reproducible vulnerabilities from the Vul4J dataset. They distinguish between *strongly coupled mutants* that fail the same tests for the same reasons as the vulnerabilities and *test coupled mutants* that fail the same tests but for different reasons. While they find the majority (32

of 45) of μ Bert-generated mutants to be strongly coupled, they also find that strongly coupled mutants are scarce, representing just 1.17% of killable mutants. It would be interesting to explore whether the use of more elaborate prompting strategies, such as those employed by *LLMorpheus* could be used to increase the ratio of strongly coupled mutants.

Tian et al. [35] consider the use of LLMs for determining whether mutants are equivalent and compare their effectiveness to that of traditional techniques for mutant equivalence detection. Their study considers the detection of equivalent mutants in 19 Java programs from the MutantBench suite [36], from which mutants were derived using standard mutation operators from μ Java [37]. Tian et al. experimented with 10 LLMs. They consider 10 state-of-the-art LLMs and several strategies for fine-tuning and prompting, and consider three widely used traditional techniques (compiler-based, ML-based, and Tree-Based Neural NetWork) as the baseline for comparison. Their results indicate that LLMs are significantly better than traditional techniques at equivalent mutant detection, with the fine-tuned code embedding strategy being the most effective. It would be interesting to explore to what extent these results carry over to detecting mutants that were produced using LLMs using tools such as *LLMorpheus*.

Similar to our interests, Wang et al. [38] perform an exploratory study on using large language models to generate mutants. Unlike our prompting strategy that generates up to three mutants per-AST node, Wang et al. explore a strategy that generates mutants at the granularity of entire methods. We demonstrate the nuances of prompt engineering in this context by exploring performance under different prompts (RQ4). These complementary works demonstrate the potential of using LLMs for mutation testing.

Several projects [39], [40] have considered the use of LLMs as mutation operators in the context of Genetic and Search-Based techniques to improve the efficiency of the search. Brownlee et al. [40] consider the generation of alternate implementations for methods and experimented with prompts exhibiting different levels of detail, similar to our experiments reported on in Section IV-F, finding that more detailed prompting generally improves the number of successful patches.

Several other works rely on LLMs to validate the results of mutation testing tools. Li and Shin [41] use 4 syntactic mutation operators and then observe the change to the natural language description that an LLM generates of the mutated code. MuTAP [42] uses an off-the-shelf syntactic mutation tool to generate mutants for a Python program and then prompts an LLM to generate a test that can detect those mutants.

Equivalent Mutants: Kushigian et al. [43] study the types and prevalence of equivalent mutants in Java programs, considering why they are equivalent and how challenging it is to detect that they are. Their study considers 19 Java open-source programs from which mutants are derived using Major [11], a rule-based mutation-testing framework for Java that supports similar mutation operators as *StrykerJS*. Their findings indicate that around 3% of mutants are equivalent, and these equivalent mutants are further classified according to criteria that reflect *why* a mutant is equivalent, and *how* this could be determined.

Based on these findings, Kushigian et al. propose Equivalent Mutant Suppression (EMS), a collection of simple static checks for detecting equivalent mutants.

Improving Mutation operators: Other approaches for mutation testing aim to generate mutants that represent a wider variety of faults. “Higher-order mutation” combines multiple mutations concurrently, creating more complex faults, but still limited by the set of operators implemented [8], [44]. More recently, Brown et al. improve mutation by mining patches for new idioms to use as mutation operators [45]. Beller et al. design a similar tool and evaluate it at Facebook, with the goal of increasing adoption of mutation testing [46]. Taking this idea further, Tufano et al. create *DeepMutation*, an approach that learns models for performing mutation from real bugs [14]. This idea was refined by Tian et al.’s *LEAM*, which improves the search process by leveraging program grammars [15]. Patra and Pradel’s *SemSeed* learns to generate mutants from fixes of real-world identifier and literal semantic bugs [16]. Unlike these approaches, *LLMorpheus* uses a *pre-trained LLM*, requiring no training to apply it to a new project.

Mutation Testing Applications and Tools: Belén Sánchez et al. [47] report on a results of a qualitative study among open-source developers on the use of mutation testing. Their findings indicate that developers find mutation testing useful for improving test suite quality, detecting bugs, and improving code maintainability and performance considerations are the biggest impediments to adoption. Much of the research advancing the state of mutation testing tooling has targeted Java, such as MuJava [37], Javalanche [48], Jumble [49], Judy [50] and Major [11]. Gopinath et al. empirically compared two of these research-oriented tools (Judy [50], Major [11]) with an industry-oriented tool (Pit [9]), finding that despite the stated similarities between the tools, each produced a somewhat different set of mutants [51]. Pit is actively maintained, and the open-source tool is also available packaged with professional plugins under the name ‘ArcMutate’ [10]. Also aimed at practitioners, the *Stryker* mutation tool is a framework that supports code written in JavaScript, TypeScript, C#, and Scala [12]. We build *LLMorpheus* atop Stryker. Deb et al. examine a new, language-agnostic approach to generating mutants using regular expressions [52]. Future work may examine the feasibility of implementing *LLMorpheus* using this approach.

Mutation and Test Generation: There is a long line of research on test-generation techniques that specifically target mutated code. DeMillo and Offutt [53] presented a technique that relies on solving systems of algebraic constraints to derive test cases that target mutated code. Fraser and Zeller [54] present μ TEST, an approach that automatically generates unit tests for object-oriented classes based on mutation analysis. Their test generation technique uses mutations as the coverage criterion that it aims to maximize and creates tests containing oracles that test the mutated value. Chekam et al. [55] present a test generation technique based on symbolic execution that systematically searches for situations where program behaviors of the original program diverges from that of mutated versions. Lee et al. [56] present a grey-box fuzzing technique that involves executing both the original and the mutated code in the

same fuzzing driver to direct the generation of test inputs toward those that kill mutants. Adapting LLM-based test generation techniques [25], [57], [58] to target mutated code would be an interesting topic for future work.

LLMs and Testing: Beyond mutation testing, LLMs have also been used for test generation. Bareiß et al. [58] present an approach for test generation that follows a few-shot learning paradigm, outperforming traditional feedback-directed test generation [59]. Tufano et al. [57] present an approach for test generation using a BART transformer model [60] that is fine-tuned on a training set of functions and corresponding tests. Lemieux et al. [29] present an approach where tests generated by Codex are used to assist search-based testing techniques [61] in situations where such techniques get “stuck” because the generated test cases diverge too far from the expected uses of the code under test. TestPilot [25] produces unit tests for JavaScript programs by prompting an LLM with the start of a test for an API function, with information about that function (signature, body, and usage examples mined from project documentation) embedded in code comments. In response, the LLM will produce a candidate test, which it executes to determine whether it passes or fails. In case of failure, TestPilot attempts to fix the failing test by re-prompting the LLM with the error message. In principle, *LLMorpheus* can be used to evaluate such test generation techniques by providing a means to assess the quality of the generated tests.

VII. CONCLUSION AND FUTURE WORK

We have presented *LLMorpheus*, an LLM-based technique for mutation testing. In this approach, code fragments at designated locations in the program’s source code are replaced with the word “PLACEHOLDER”, and an LLM is given a prompt that includes: general background on mutation testing, the original code fragment, and instructions directing the LLM to replace the placeholder with a buggy piece of code. The mutants produced by *LLMorpheus* are passed to a modified version of the popular *StrykerJS* mutation testing tool, which runs the tests, classifies mutants, and creates an interactive web page for inspecting the results.

An empirical evaluation on 13 subject applications demonstrates that *LLMorpheus* can produce mutants that resemble real bugs that cannot be produced using standard mutation operators. We found that the majority (80%) of surviving mutants produced by *LLMorpheus* are behavioral changes and that 20% of them are equivalent mutants. Experiments with variations on the prompt template reveal that the “full” template that includes all information performs best and that omitting parts of the information from this template matters to varying degrees. From experiments with five LLMs, we found that *llama-3.3-70b-instruct* and *codellama-34b-instruct* generally produced the largest number of mutants and surviving mutants. Moreover, in a case study involving 40 real-world bugs, we found that *LLMorpheus* produced mutants that are syntactically identical to the buggy code fragments in 10 cases and mutants that produce the same test failures as the original bug in an additional 26 cases. These results provide strong evidence that *LLMorpheus*

is capable of generating mutants whose behavior resembles that of real-world bugs and that this capability is not entirely due to training-set leakage.

The number of mutants produced by *LLMorpheus* can become quite large, and executing them can take considerable time. In future work, we plan to explore techniques for pruning and prioritizing mutants, focusing particularly on reducing the number of equivalent mutants. From a manual investigation of 105 equivalent mutants, we observed several common patterns, such as replacing an condition `!x` with `x === null` or `x === undefined` or replacing call to `String.substring` with calls to `String.substr` and `String.slice`, two methods with similar semantics. We expect that most of these equivalent mutants can be filtered out using simple AST-based analysis. However, further investigation is needed because some mutants that cause behavioral differences are syntactically similar to these patterns. This means that any pattern-matching-based approach should consider the context of the mutation to determine whether a mutant is likely to be equivalent. To deal with more challenging cases, future work could also explore the use of symbolic execution or efficient formal reasoning techniques for automatically identifying mutants that are likely to be equivalent.

LLMorpheus currently employs a fixed strategy for introducing placeholders, as illustrated in Fig. 6. While this strategy has been shown to be effective at producing large numbers of (surviving) mutants and at producing mutants that resemble real-world bugs, it precludes the creation of mutants at locations that do not match this strategy. As future work, we plan to explore mechanisms that allow users to specify a placeholder scheme, e.g., as a predicate on AST nodes.

In our research, we used LLMs in their default configuration without any fine-tuning. The strong results obtained with the relatively small *codellama-34b-instruct* LLM that is trained for code-related tasks suggests that fine-tuning an LLM for the specific task of mutation testing might be worthwhile, particularly to optimize the number of mutants that are not equivalent.

ACKNOWLEDGMENT

Part of the work described in this paper was done while the first author was on sabbatical with GitHub. Frank Tip holds concurrent appointments as a Professor of Computer Science at Northeastern University and as an Amazon Scholar. The paper describes work performed at Northeastern University and is not associated with Amazon.

REFERENCES

- [1] G. Petrović, M. Ivanković, G. Fraser, and R. Just, “Does mutation testing improve testing practices?” in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, 2021, pp. 910–921.
- [2] G. Petrović, M. Ivanković, G. Fraser, and R. Just, “Practical mutation testing at scale: A view from google,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 10, pp. 3900–3912, Oct. 2022.
- [3] G. Petrović, M. Ivanković, G. Fraser, and R. Just, “Please fix this mutant: How do developers resolve mutants surfaced during code review?” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2023, pp. 150–161.
- [4] R. DeMillo, R. Lipton, and F. Sayward, “Hints on test data selection: Help for the practicing programmer,” *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

- [5] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng. (FSE)*, 2014, pp. 654–665.
- [6] T. Laurent, S. Gaffney, and A. Ventresque, "Re-visiting the coupling between mutants and real faults with defects4j 2.0," in *Proc. IEEE Int. Conf. Softw. Testing, Verification Validation Workshops (ICSTW)*, 2022, pp. 189–198.
- [7] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, Sep./Oct. 2011.
- [8] A. S. Ghiduk, M. R. Girgis, and M. H. Shehata, "Higher order mutation testing: A systematic literature review," *Comput. Sci. Rev.*, vol. 25, pp. 29–48, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013716301095>
- [9] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "Pit: a practical mutation testing tool for java (demo)," in *Proc. 25th Int. Symp. Softw. Test. Anal. (ISSTA)*, New York, NY, USA: ACM, 2016, pp. 449–452. [Online]. Available: <https://doi.org/10.1145/2931037.2948707>
- [10] H. Coles, "Arcmutate: Advanced mutation testing for Java and Kotlin," 2024. Accessed: Mar. 1, 2025. [Online]. Available: <https://www.arcmutate.com>
- [11] R. Just, "The Major mutation framework: efficient and scalable mutation analysis for java," in *Proc. Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2014, pp. 433–436, doi: 10.1145/2610384.2628053.
- [12] Stryker Team, "Stryker mutator," Accessed: Jan. 8, 2025. [Online]. Available: <https://stryker-mutator.io>
- [13] H. Coles, "Arcmutate: Extended mutation operators," 2024. Accessed: Mar. 1, 2025. [Online]. Available: <https://docs.arcmutate.com/docs/extended-operators.html>
- [14] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "Learning how to mutate source code from bug-fixes," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, 2019, pp. 301–312.
- [15] Z. Tian, J. Chen, Q. Zhu, J. Yang, and L. Zhang, "Learning to construct better mutation faults," in *Proc. 37th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2023, doi: 10.1145/3551349.3556949.
- [16] J. Patra and M. Pradel, "Semantic bug seeding: a learning-based approach for creating realistic bugs," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng., (ESEC/FSE)*, 2021, pp. 906–918.
- [17] "zip-a-folder," 2025. Accessed: Jan. 8, 2025. [Online]. Available: <https://github.com/maugenst/zip-a-folder>
- [18] R. Just and F. Schweiggert, "Higher accuracy and lower run time: Efficient mutation analysis using non-redundant mutation operators," *Softw. Testing, Verification Rel. (JSTVR)*, vol. 25, nos. 5–7, pp. 490–507, Jan. 2015.
- [19] R. Just, B. Kurtz, and P. Ammann, "Inferring mutant utility from program context," in *Proc. Int. Symp. Softw. Test. Anal. (ISSTA)*, Santa Barbara, CA, USA, Jul. 2017, pp. 284–294.
- [20] "countries-and-timezones," Accessed: Jan. 8, 2025. [Online]. Available: <https://github.com/manuelmhr/countries-and-timezones>
- [21] M. Madsen, F. Tip, and O. Lhoták, "Static analysis of event-driven Node.js JavaScript applications," in *Proc. ACM SIGPLAN Int. Conf. Object-Oriented Program. Syst., Lang., Appl., (OOPSLA)*, Pittsburgh, PA, USA, 2015, J. Aldrich and P. Eugster, Eds., New York, NY, USA: ACM, 2015, pp. 505–519.
- [22] E. Arteca, M. Schäfer, and F. Tip, "Learning how to listen: Automatically finding bug patterns in event-driven JavaScript APIs," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 166–184, Jan. 2023.
- [23] "Babel," Accessed: Jan. 8, 2025. [Online]. Available: <https://babeljs.io/>
- [24] "Handlebars," Accessed: Jan. 8, 2025. [Online]. Available: <https://handlebarsjs.com/>
- [25] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using Large Language Models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
- [26] G. Gay and A. Salahirad, "How closely are common mutation operators coupled to real faults?" in *Proc. IEEE Conf. Softw. Test., Verification Validation (ICST)*, 2023, pp. 129–140.
- [27] P. Gyimesi et al., "BugJS: A benchmark of javascript bugs," in *Proc. 12th IEEE Int. Conf. Softw. Testing, Verification Validation (ICST)*, 2019.
- [28] A. Shi, J. Bell, and D. Marinov, "Mitigating the effects of flaky tests on mutation testing," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2019, pp. 112–122.
- [29] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. 45th Int. Conf. Softw. Eng.*, 2023, pp. 919–931.
- [30] M. L. McHugh, "Interrater reliability: the kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.
- [31] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, "Mutation testing advances: An analysis and survey," *Adv. Comput.*, vol. 112, pp. 275–378, 2019.
- [32] R. Degiovanni and M. Papadakis, "µBERT: Mutation testing using pre-trained language models," in *Proc. 15th IEEE Int. Conf. on Softw. Testing, Verification and Validation Workshops ICST Workshops*, Valencia, Spain, 2022, Piscataway, NJ, USA: IEEE Press, 2022, pp. 160–169.
- [33] A. Khanfir, R. Degiovanni, M. Papadakis, and Y. L. Traon, "Efficient mutation testing via pre-trained language models," 2023, *arXiv:2301.03543*.
- [34] A. Garg, R. Degiovanni, M. Papadakis, and Y. L. Traon, "On the coupling between vulnerabilities and LLM-generated mutants: A study on Vul4J dataset," in *IEEE Conference on Softw. Testing, Verification Validation (ICST)*, 2024, Toronto, ON, Canada, 2024, Piscataway, NJ, USA: IEEE Press, 2024, pp. 305–316.
- [35] Z. Tian, H. Shu, D. Wang, X. Cao, Y. Kamei, and J. Chen, "Large language models for equivalent mutant detection: How far are we?" in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2024, Vienna, Austria, M. Christakis and M. Pradel, Eds., 2024, pp. 1733–1745.
- [36] L. van Hijfte and A. Oprescu, "Mutantbench: An equivalent mutant problem comparison framework," in *Proc. 14th IEEE Int. Conf. Softw. Testing, Verification Validation Workshops, ICST Workshops, Porto de Galinhas*, Brazil, Piscataway, NJ, USA: IEEE Press, 2021, pp. 7–12.
- [37] Y.-S. Ma, J. Offutt, and Y.-R. Kwon, "MuJava: a mutation system for Java," in *Proc. 28th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2006, pp. 827–830.
- [38] B. Wang, M. Chen, Y. Lin, M. Papadakis, and J. M. Zhang, "An exploratory study on using large language models for mutation testing," 2024, *arXiv:2406.09843*.
- [39] S. Kang and S. Yoo, "Towards objective-tailored genetic improvement through Large Language Models," in *Proc. IEEE/ACM Int. Workshop Genet. Improvement, Gi@Icse 2023*, Melbourne, Australia, Piscataway, NJ, USA: IEEE Press, 2023, pp. 19–20.
- [40] A. E. I. Brownlee et al., "Enhancing genetic improvement mutations using Large Language Models," in *Proc. Search-Based Softw. Eng. 15th Int. Symp. (SSBSE)*, San Francisco, CA, USA, P. Arcaini, T. Yue, and E. M. Fredericks, Eds., vol. 14415, New York, NY, USA: Springer, 2023, pp. 153–159.
- [41] Z. Li and D. Shin, "Mutation-based consistency testing for evaluating the code understanding capability of LLMs," in *Proc. IEEE/ACM 3rd Int. Conf. AI Eng. Softw. Eng., AI, CAIN 2024*, Lisbon, Portugal, J. Cleland-Huang, J. Bosch, H. Muccini, and G. A. Lewis, Eds., New York, NY, USA: ACM, 2024, pp. 150–159.
- [42] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained Large Language Models and mutation testing," *Inf. Softw. Technol.*, vol. 171, 2024, Art. no. 107468.
- [43] B. Kushigian, S. J. Kaufman, R. Featherman, H. Potter, A. Madadi, and R. Just, "Equivalent mutants in the wild: Identifying and efficiently suppressing equivalent mutants for Java programs," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA)*, Vienna, Austria, 2024, M. Christakis and M. Pradel, Eds., New York, NY, USA: ACM, 2024, pp. 654–665.
- [44] Y. Jia and M. Harman, "Higher order mutation testing," *Inf. Softw. Technol.*, vol. 51, no. 10, pp. 1379–1393, 2009.
- [45] D. B. Brown, M. Vaughn, B. Liblit, and T. Reps, "The care and feeding of wild-caught mutants," in *Proc. 11th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2017, pp. 511–522, doi: 10.1145/3106237.3106280.
- [46] M. Beller et al., "What it would take to use mutation testing in industry: a study at facebook," in *Proc. 43rd Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 268–277.
- [47] A. Belén Sánchez, J. A. Parejo, S. Segura, A. D. Toro, and M. Papadakis, "Mutation testing in practice: Insights from open-source software developers," *IEEE Trans. Softw. Eng.*, vol. 50, no. 5, pp. 1130–1143, May 2024, doi: 10.1109/TSE.2024.3377378.
- [48] D. Schuler and A. Zeller, "Javalanche: efficient mutation testing for Java," in *Proc. 7th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2009, pp. 297–298.

- [49] S. A. Irvine, T. Pavlinic, L. Trigg, J. G. Cleary, S. Inglis, and M. Utting, "Jumble Java byte code to measure the effectiveness of unit tests," in *Proc. Testing: Acad. Ind. Conf. Practice Res. Techn. MUTATION (TAICPART-MUTATION)*, Los Alamitos, CA, USA: IEEE Computer Soc., 2007, pp. 169–175.
- [50] L. Madeyski, "Judy – a mutation testing tool for Java," *IET Softw.*, vol. 4, pp. 32–42, 10 February 2010. [Online]. Available: <https://digital-library.theiet.org/content/journals/10.1049/iet-sen.2008.0038>.
- [51] R. Gopinath, I. Ahmed, M. A. Alipour, C. Jensen, and A. Groce, "Does choice of mutation tool matter?" *Softw. Qual. J.*, vol. 25, no. 3, pp. 871–920, 2017.
- [52] S. Deb, K. Jain, R. Van Tonder, C. L. Goues, and A. Groce, "Syntax is all you need: A universal-language approach to mutant generation," in *Proc. ACM Softw. Eng. (FSE)*, 2024, pp. 654–674.
- [53] R. A. DeMillo and A. J. Offutt, "Constraint-based automatic test data generation," *IEEE Trans. Softw. Eng.*, vol. 17, no. 9, pp. 900–910, 1991. [Online]. Available: <https://doi.org/10.1109/32.92910>.
- [54] G. Fraser and A. Zeller, "Mutation-driven generation of unit tests and oracles," in *Proceedings of the Nineteenth International Symposium on Software Testing and Analysis, ISSTA 2010, Trento, Italy July 12-16, 2010*, P. Tonella and A. Orso, Eds., New York, NY, USA: ACM, 2010, pp. 147–158.
- [55] T. T. Chekam, M. Papadakis, M. Cordy, and Y. L. Traon, "Killing stubborn mutants with symbolic execution," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 2, pp. 1–19–23, 2021.
- [56] J. Lee, E. Viganò, O. Cornejo, F. Pastore, and L. C. Briand, "Fuzzing for CPS mutation testing," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Luxembourg*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1377–1389.
- [57] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," May 2021. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/unit-test-case-generation-with-transformers-and-focal-context/>
- [58] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code generation tools (almost) for free? A study of few-shot, pre-trained language models on code," 2022, *arXiv:2206.01335*.
- [59] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for java," in *Companion to the 22Nd ACM SIGPLAN Conf. Object-Oriented Programm. Syst. Appl. Companion (OOPSLA)*, New York, NY, USA: ACM, 2007, pp. 815–816.
- [60] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv:1910.13461*.
- [61] A. Panichella, F. M. Kifetew, and P. Tonella, "Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets," *IEEE Trans. Softw. Eng.*, vol. 44, no. 2, pp. 122–158, Feb. 2018.



Frank Tip received the Ph.D. degree from the University of Amsterdam, Amsterdam, Netherlands, in 1995. Currently, he is a Professor with Khoury College of Computer Sciences, Northeastern University, Boston, MA, USA. His research interests include static and dynamic program analysis, asynchronous programming, refactoring, and testing.



Jonathan Bell received the Ph.D. degree from Columbia University, New York, in 2016. Currently, he is an Associate Professor with Khoury College of Computer Sciences, Northeastern University, Boston, MA, USA. His research interests include software testing, dynamic analysis, and software systems.



Max Schäfer received the Ph.D. degree in computer science from the University of Oxford, in 2010. Currently, he is working as an AI Researcher with XBOW. His research interests include program analysis, test generation and security, employing both traditional and AI-based methods.